Introduction to Applied Statistics for STEM Researchers

Frank Hause

Research Training Group 2467: „Intrinsically Disordered Proteins – Molecular Principles, Cellular Functions, and Diseases“

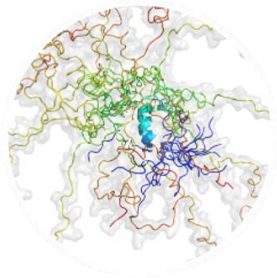
Winter Semester 2025/26

MARTIN-LUTHER-UNIVERSITÄT
HALLE-WITTENBERG

RTG 2467

Intrinsically Disordered Proteins – Molecular Principles, Cellular Functions, and Diseases

Speaker of the RTG: Prof. Dr. Andrea Sinz
Scientific Coordinator: Dr. Oleksandr Sorokin

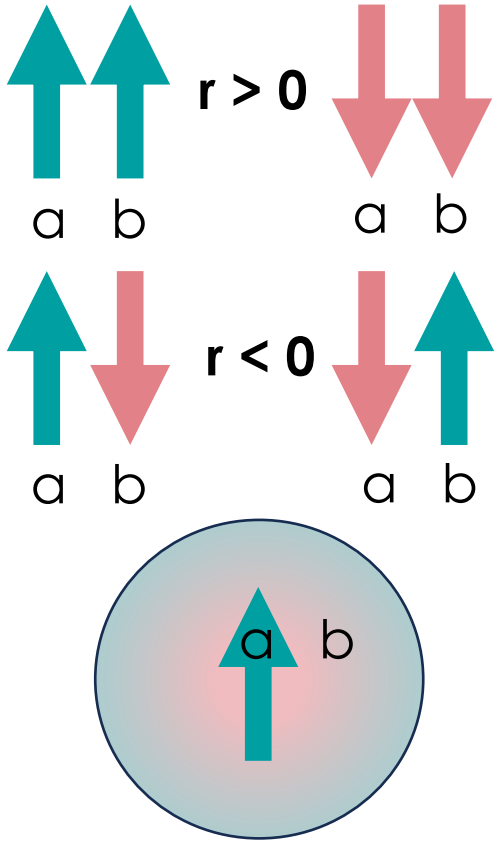


Funded by
DFG Deutsche Forschungsgemeinschaft
German Research Foundation

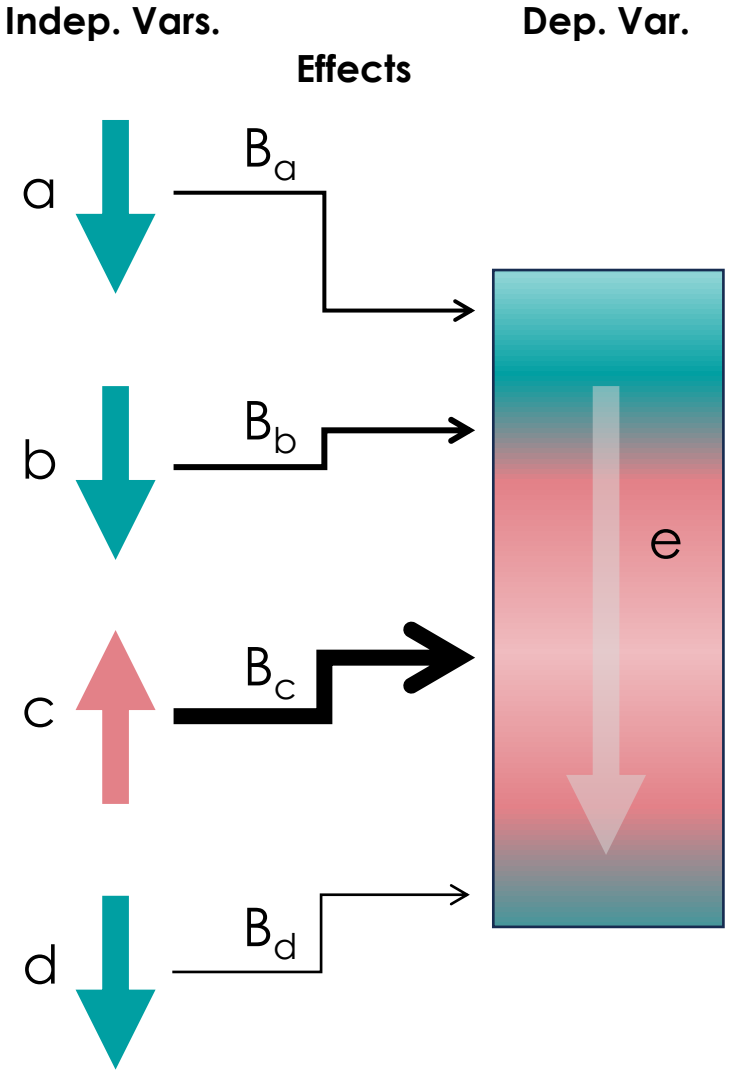
Recap

Correlations

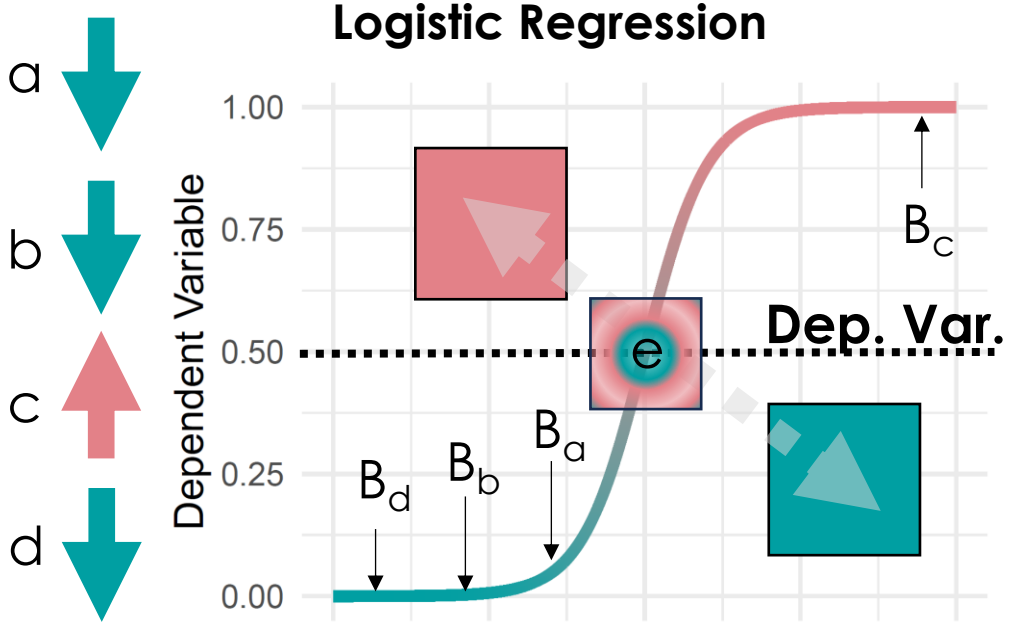
(Spearman, Pearson, Partial)



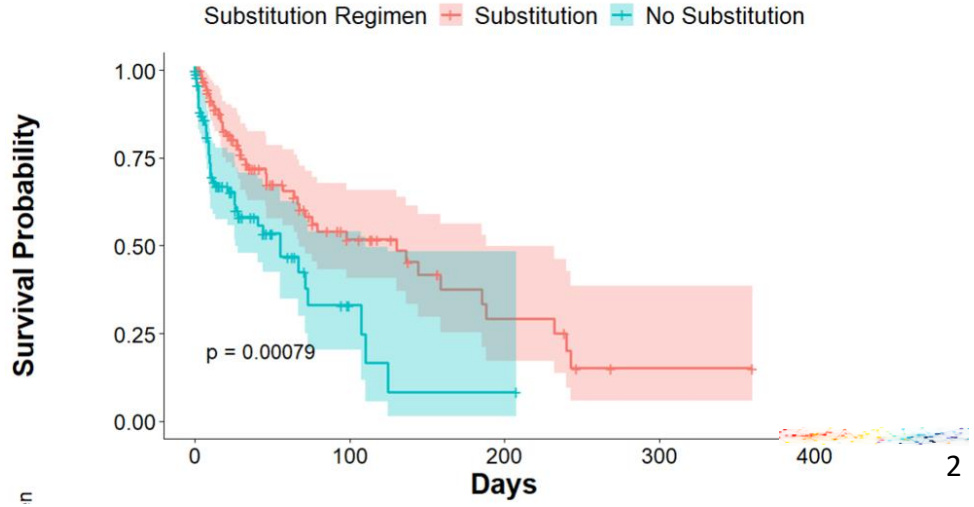
Linear Regression



Logistic Regression



Survival Analysis



Course Outline

Part I: Comparing Entities

Part II: Inferring Causality

Part III: Coping with Complexity

Scripts & Exercises:

frank.hause@medizin.uni-halle.de

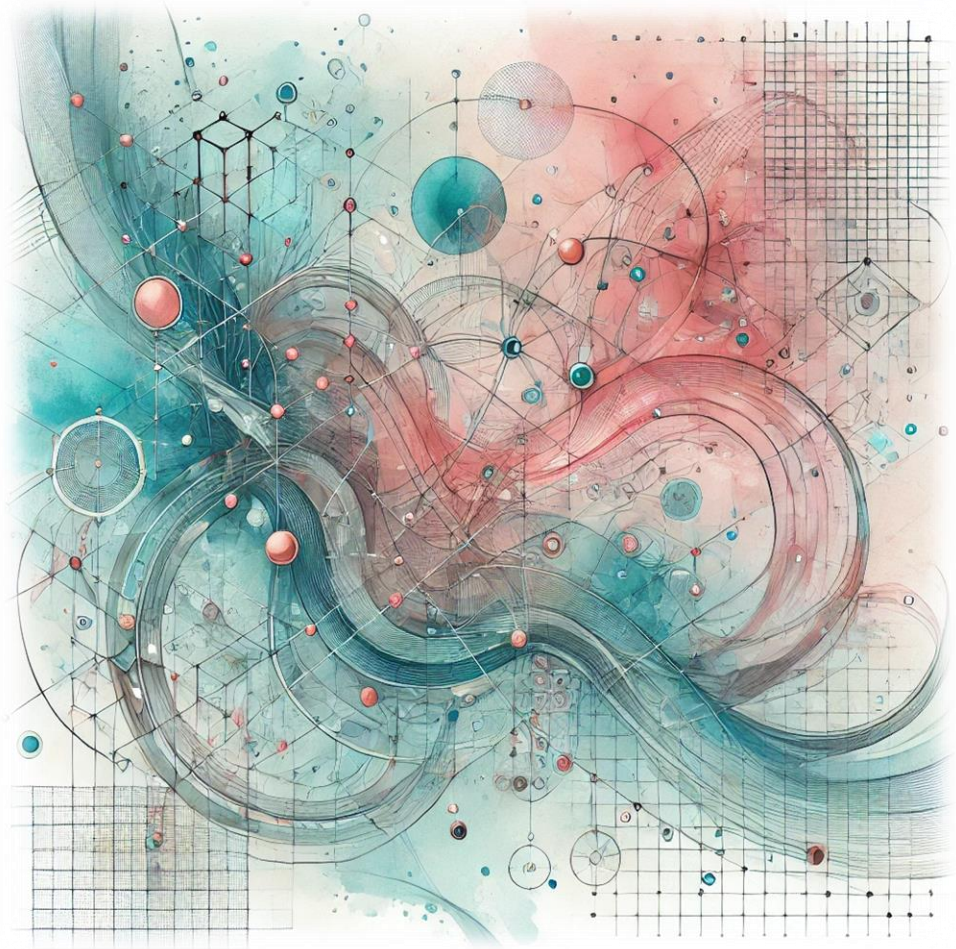
@fhause.bsky.social

Part III: Coping with Complexity

Part III-1: Outline

- **Introduction to Complexity**
- **Data Normalization**
  - Z Score Transformation
  - Log2() Transformation
- **P Value Adjustment**
  - Bonferroni Correction
  - Benjamini-Hochberg Correction/False Discovery Rate
- **Complexity Reduction**
  - Unsupervised: Principal Component Analysis (PCA)
  - Supervised: Partial Least Squares—Discriminant Analysis (PLS-DA)
- **Clustering Algorithms I:**
  - "Non-Parametric" Non-Deterministic Clustering: Hierarchical Clustering
  - "Parametric" Deterministic Classification: *k*-Means
- **Comparing High-Dimensional Entities**: Massive Parallel Hypothesis Testing (LIMMA)
- ***Excursus:*** Missingness and Data Imputation

Complex Data



- **Complex Systems**: deterministic, non-linear, interdependent, unpredictable (chaotic), emergent properties

- **Dogma:** Data containing a 'hidden layer ' of information that is only accessible with appropriate statistical means → high arguable

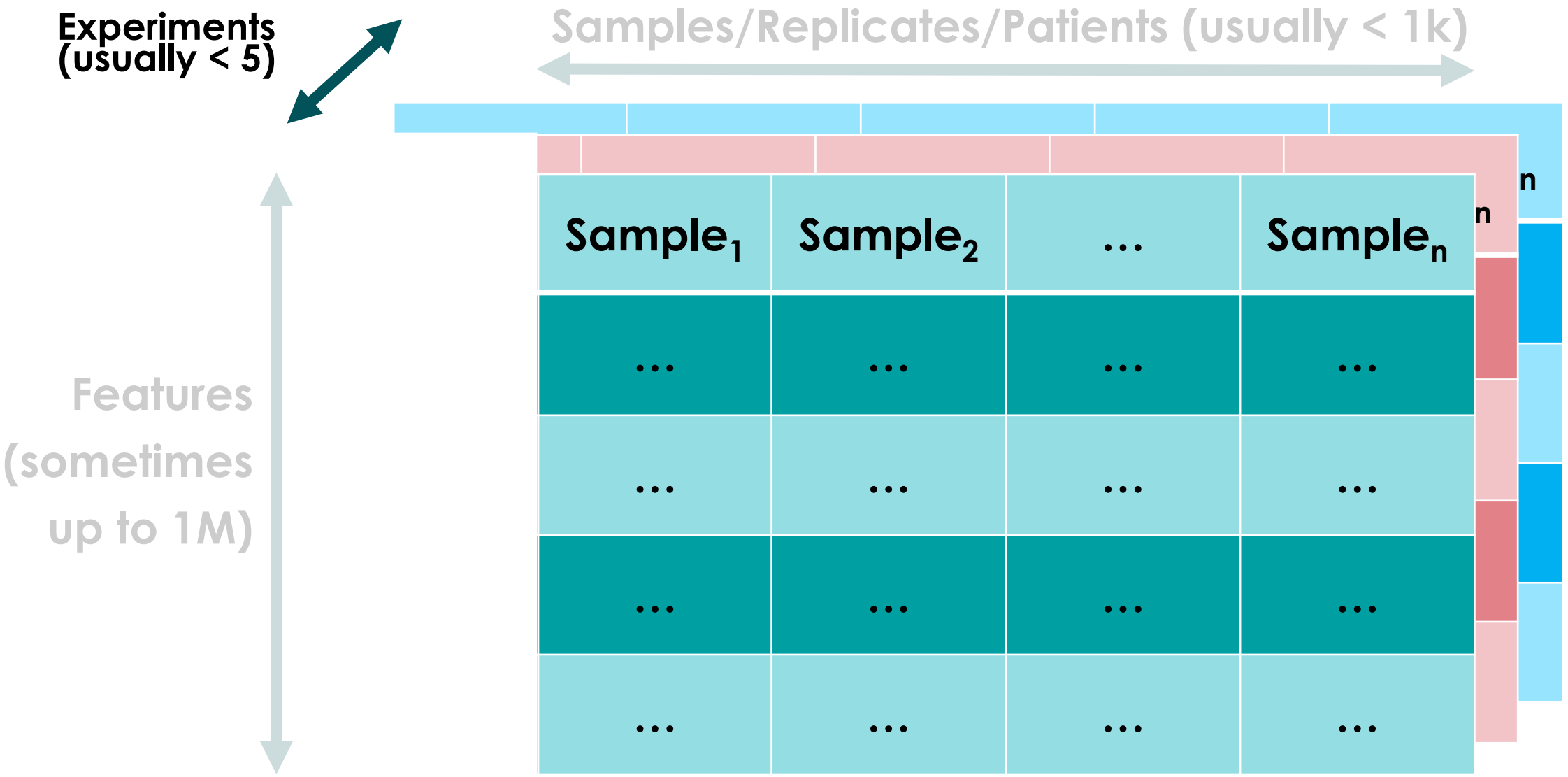
→Data are **massive** (very, very huge amount)

→Data from different experiments must be integrated (**dimensionality**)

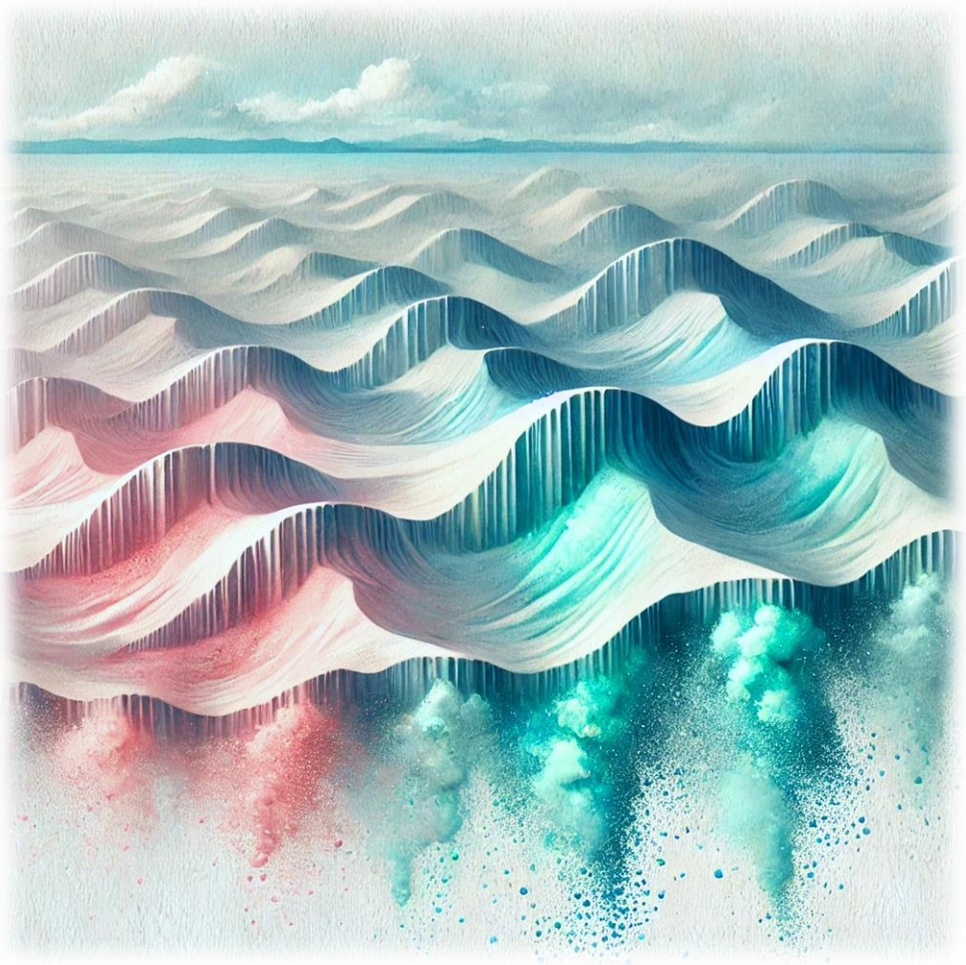
→Data are highly **inter-correlating**

→Requirements of classical statistical tests will **not be met by this type of data**

Complex Data



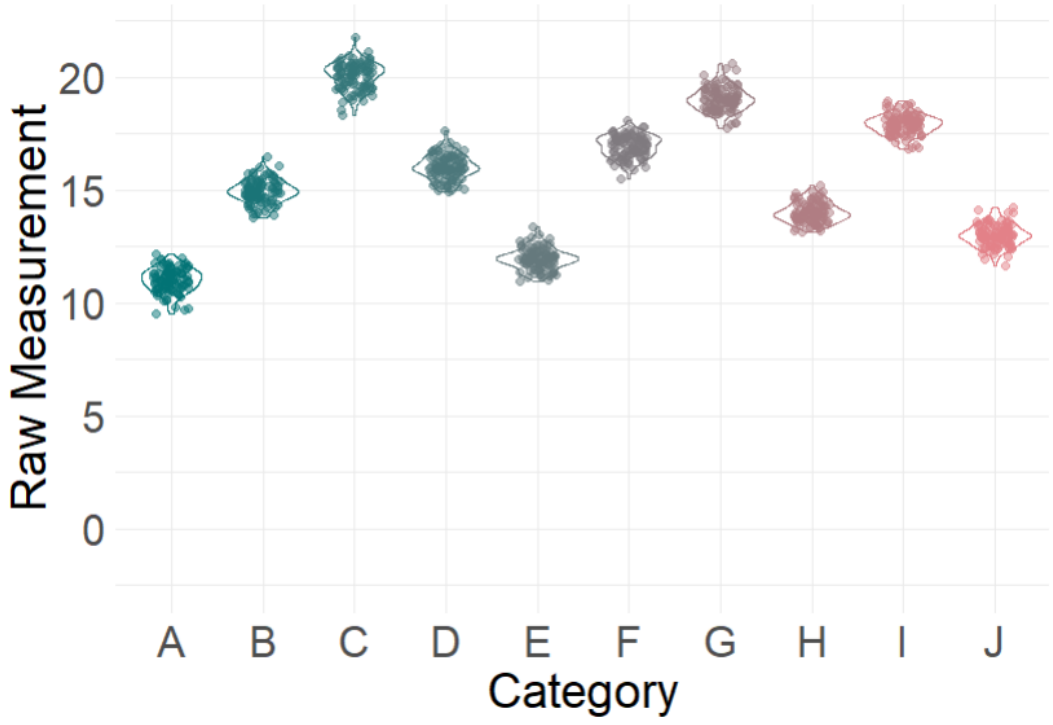
Data Normalization



- Z-Score Normalization

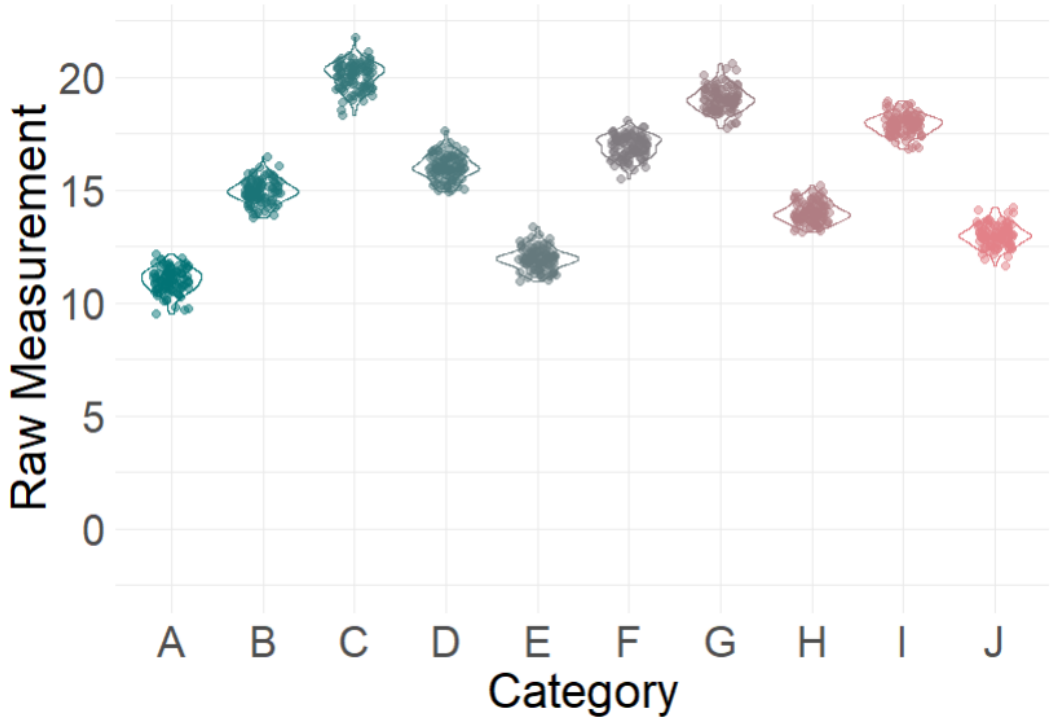
- Log2() Normalization

Z-Score Normalization



- Raw measurements are often not comparable due to **varying means and variances**
- **Z-score transformation standardizes** these values for consistent interpretation
- Rescales measurements to have a **mean of 0** and a **standard deviation of 1**

Z-Score Normalization

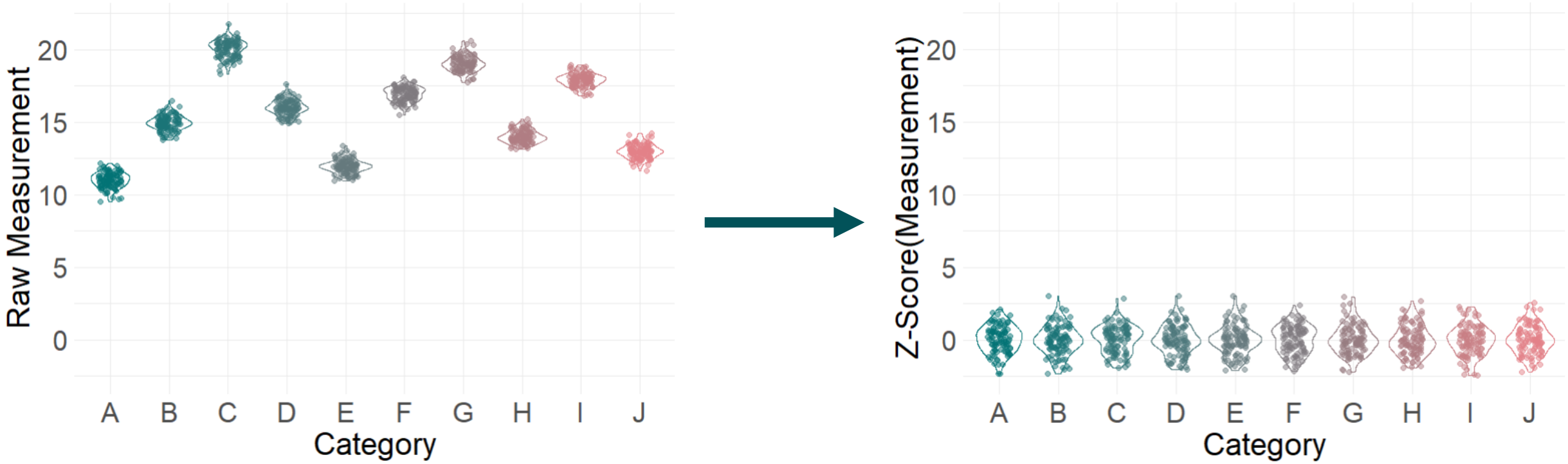


$$Z_i = \frac{X_i - \mu_{1..n}}{\sigma_{1..n}}$$

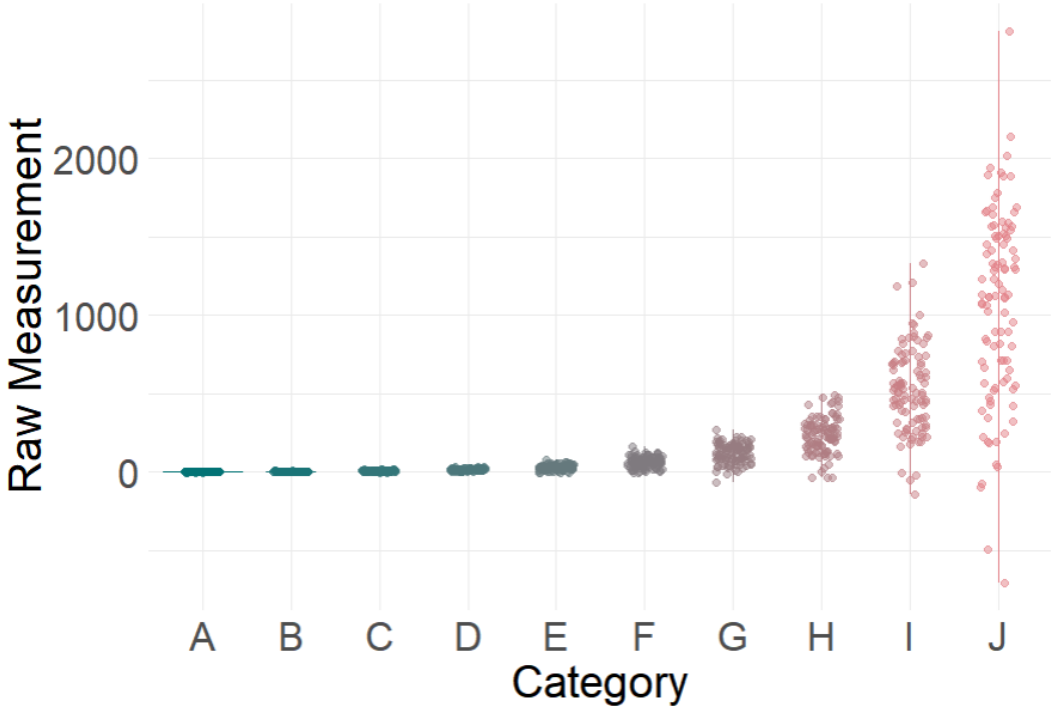
X_i … Single Raw Value in a Cohort
$\mu_{1..n}$ … Average of this Cohort
$\sigma_{1..n}$ … Std. Deviation of this Cohort
Z_i … Single Z-Score Transformed Value

```
Z_Score = scale(Measurements)
```

Z-Score Normalization

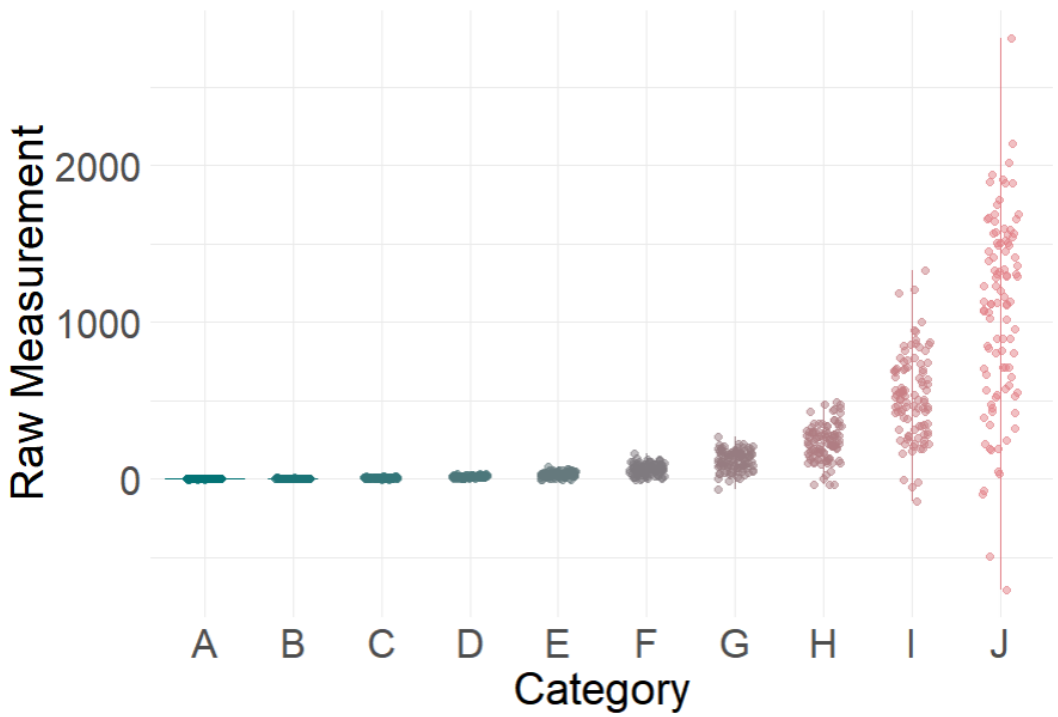


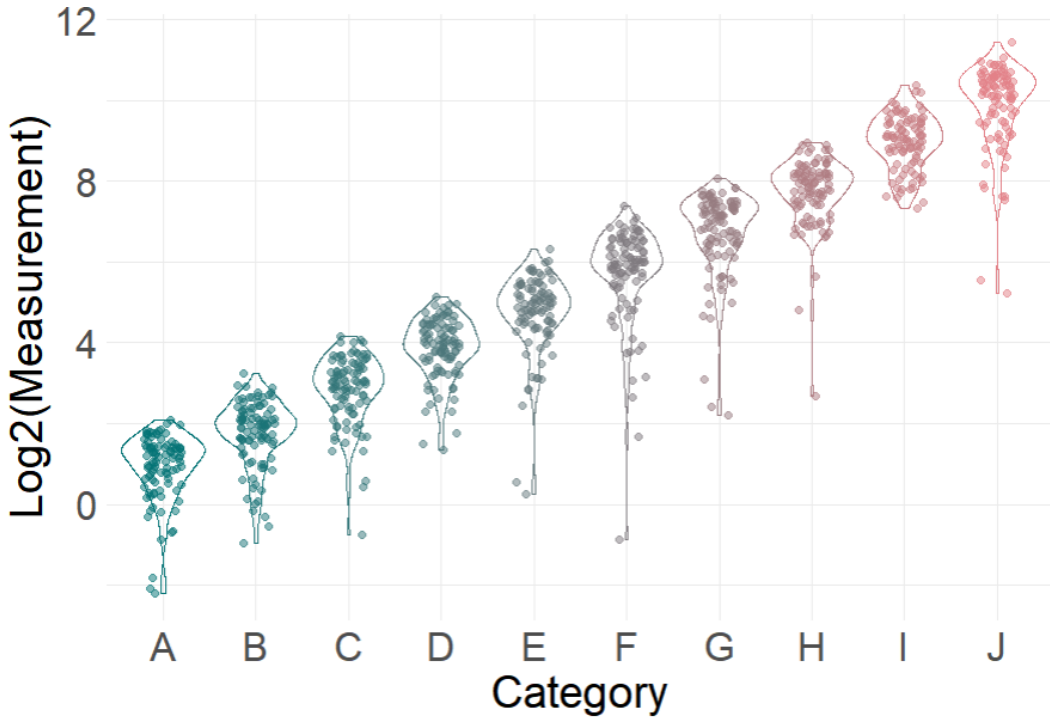
Log2() Normalization



- Converts exponential or **multiplicative data** into an **additive scale**
- Reduces skewness by compressing large values → yields almost **normal distributions**
- Allows better visualization and interpretation

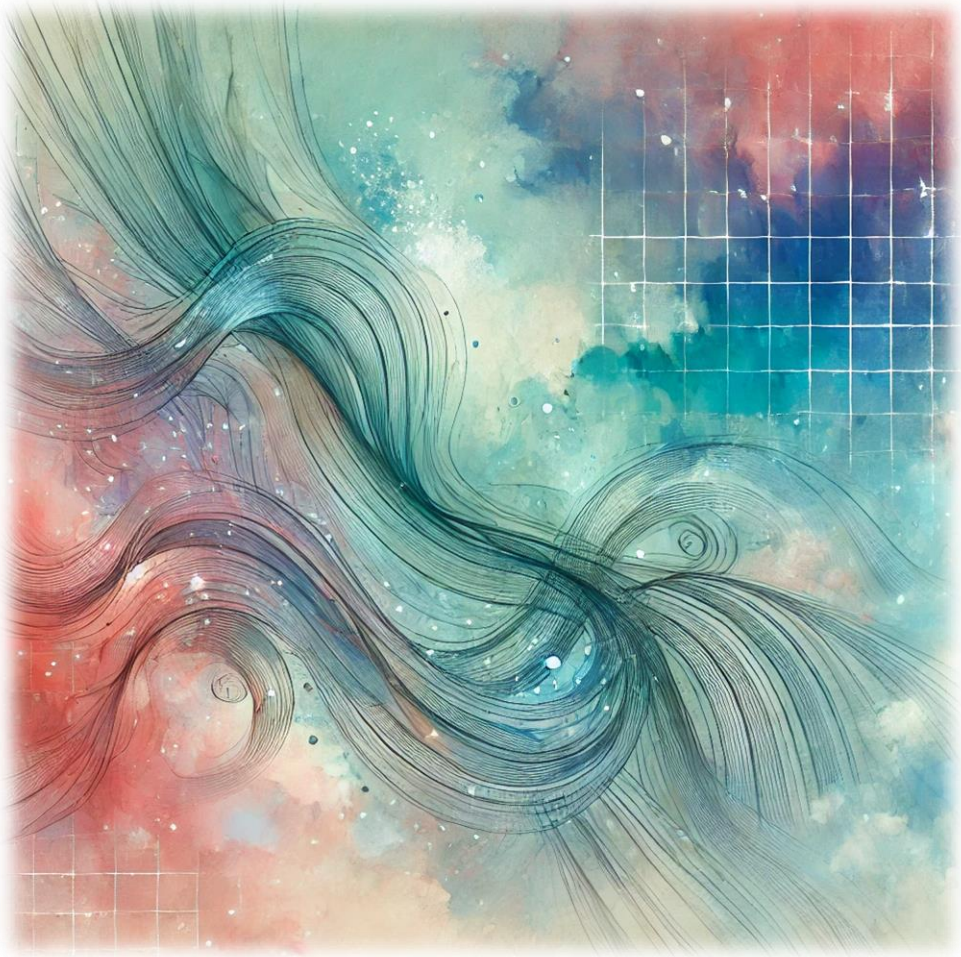
Log2() Normalization






```
log2Normalized = log2(Measurements)
```

P Value Adjustment



- Bonferroni Correction

- Benjamini-Hochberg Correction

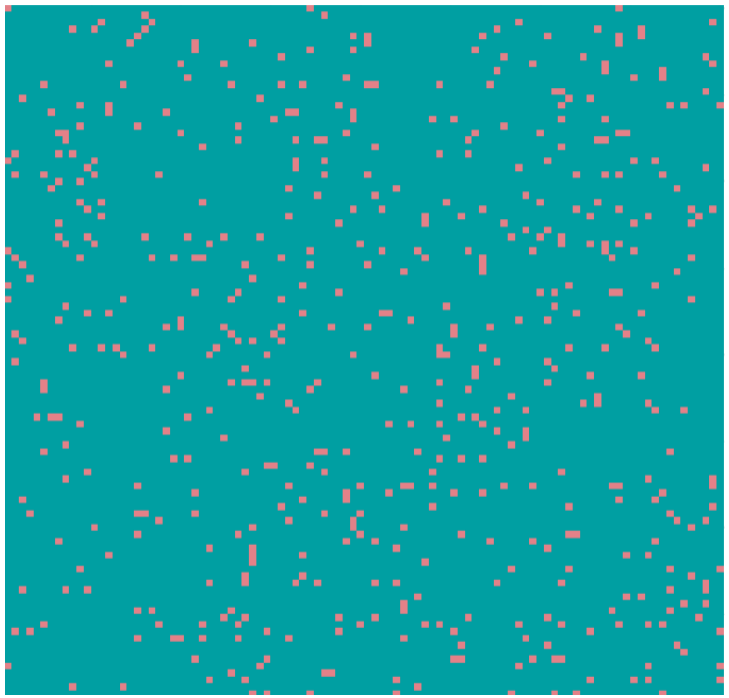
P Value Adjustment

p = 0.05

n = 1

n = 100

n = 10,000



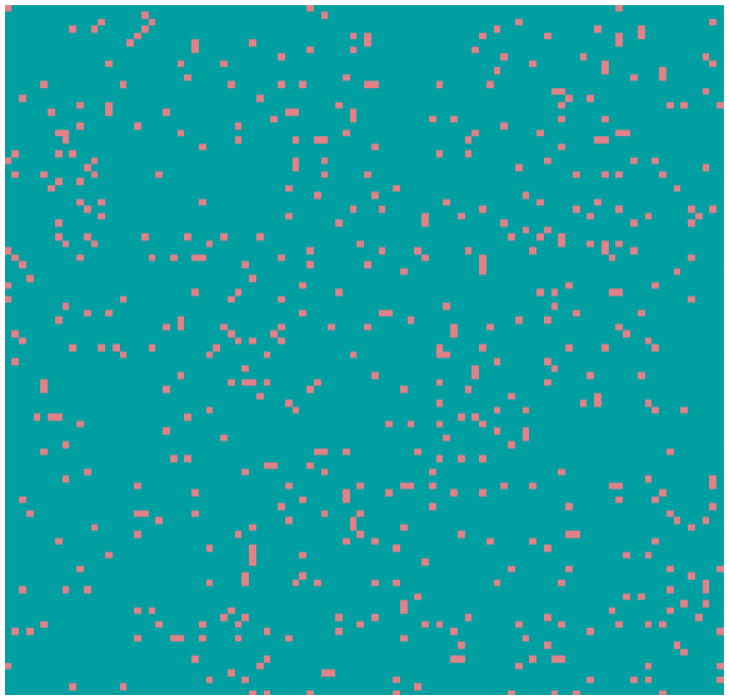
P Value Adjustment

- Necessary to control **Type I error** (falsely accepting a non-significant difference) rate in multiple testing

- Adjusts raw p-values to **account for multiple comparisons**

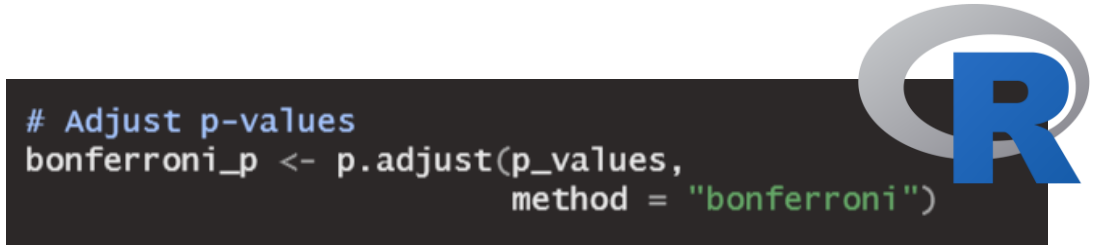
- Two main approaches: **Family-Wise Error Rate** (FWER) control and **False Discovery Rate** (FDR) control.

n = 10,000



Bonferroni Correction

- **Method**: Adjusts p-values by multiplying them by the number of tests (p_adj = p * m).
- **Stringent**: Controls family-wise error rate (FWER), reducing the chance of false positives.
- **Drawback**: Too conservative, increasing false negatives (missed true discoveries).
- **Best suited for**: Small numbers of hypotheses or when false positives are costly (e.g., clinical trials).



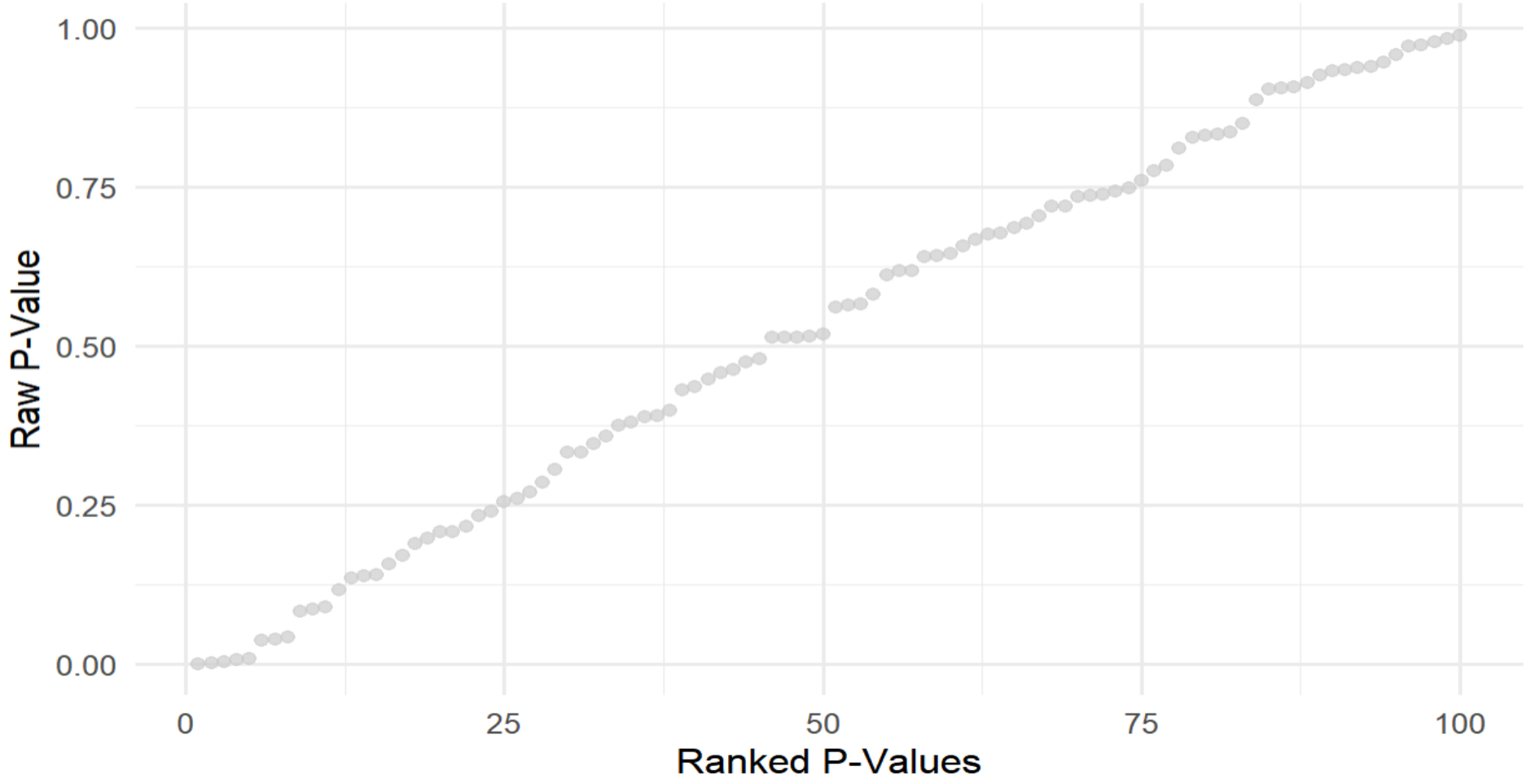
```r
# Adjust p-values
bonferroni_p <- p.adjust(p_values,
                         method = "bonferroni")
```

Benjamini-Hochberg Correction

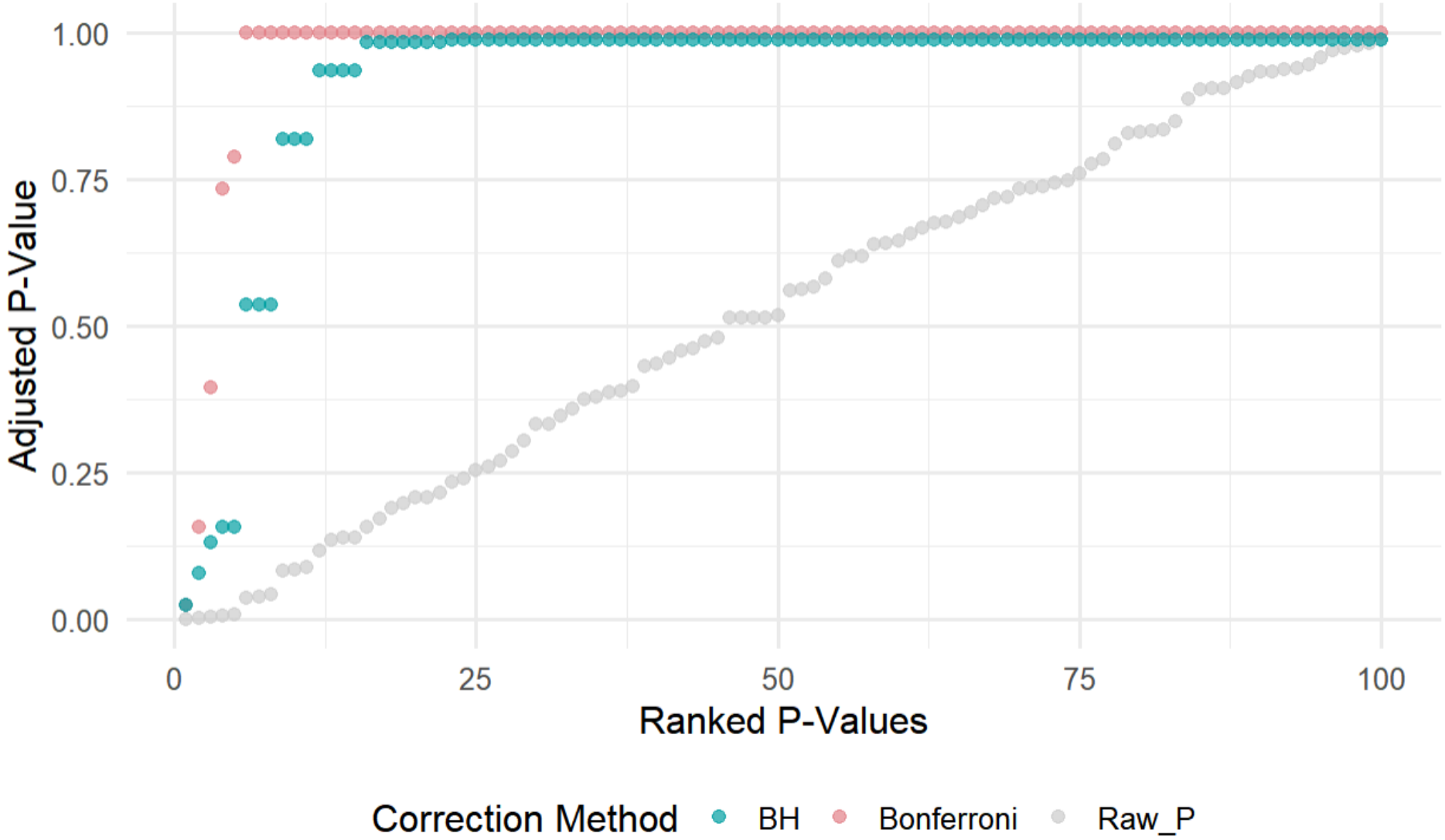
- **Method:** Rank p-values in ascending order, then adjust based on rank (p_adj = (p * m) / rank).
- **Less stringent than Bonferroni**: Controls False Discovery Rate (FDR) instead of FWER.
- **Compromise**: Allows some false positives but prioritizes true discoveries.
- **Best suited for**: High-throughput data (e.g., omics studies like RNA-seq, microarrays, quantitative proteomics, etc.).

```r
# Adjust p-values
bh_p <- p.adjust(p_values,
                 method = "BH")
```

Bonferroni versus Benjamini-Hochberg



Bonferroni versus Benjamini-Hochberg



Complexity Reduction

- Multi-Dimensional Data

- PCA

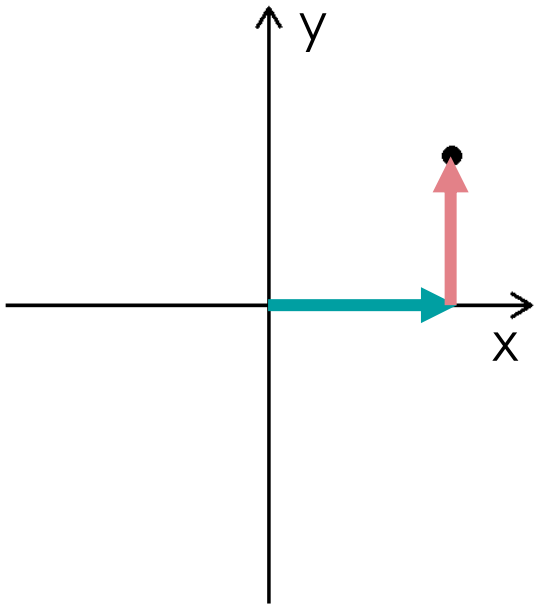
- (s)PLS-DA

Multi-Dimensional Data

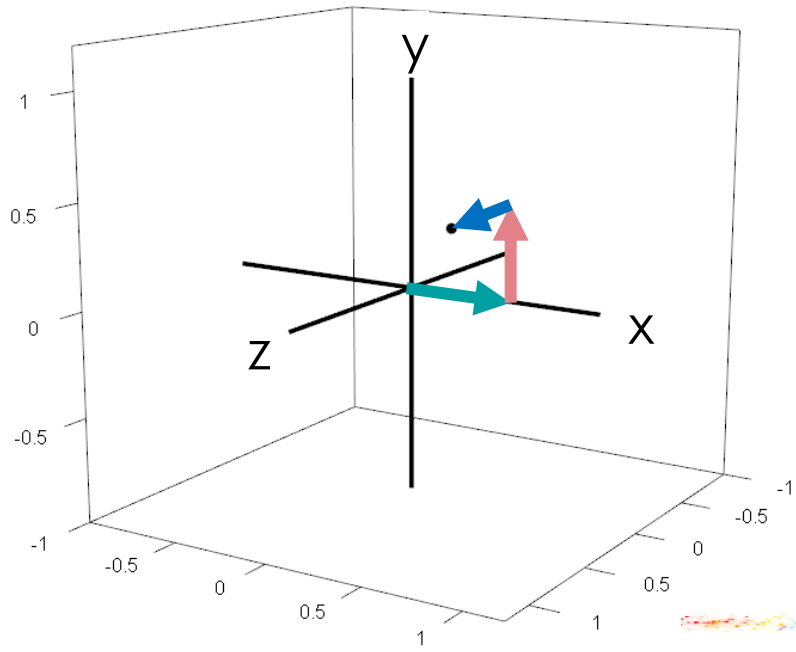
```
# Example point coordinates
example_point <- list(x = 0.7,
                      y = 0.5,
                      z = 0.8)
```

1D

2D



3D



Multi-Dimensional Data

	$Sample_1$	$Sample_2$	…	$Sample_n$
$Feature_1$	…	…	…	…
$Feature_2$	…	…	…	…
…	…	…	…	…
$Feature_p$	…	…	…	…

- Usually data has as many dimensions as it has **describing features**
- Each sample can be described as point in an ***n*-dimensional Euclidean space**
- Groups of points form **higher-dimensional objects**
- Task: **reducing dimensionality** in a way that allows for describing ***n*-dimensional research objects on two-dimensional paper**

Principal Component Analysis

	Variable1	Variable2	Variable3	Variable4
Sample_1	3.879049	3.5694592	6.235293	4.810022
Sample_2	4.539645	2.2465148	4.105051	11.789556
Sample_3	8.117417	2.5001889	5.018885	6.951049
Sample_4	5.141017	1.4721369	5.487816	5.403461
Sample_5	5.258575	1.3923132	9.687724	7.291161
Sample_6	8.430130	3.4552930	4.696100	7.408472
Sample_7	5.921832	3.6723147	6.470773	11.329761
Sample_8	2.469878	3.0795063	6.155922	8.254212
Sample_9	3.626294	4.3834012	4.076287	10.262161
Sample_10	4.108676	6.0751270	5.857384	6.502124
Sample_11	7.448164	2.2634533	8.889102	8.643336
Sample_12	5.719628	-0.4637533	6.903008	7.025942
Sample_13	5.801543	4.5086078	6.082466	8.283751
Sample_14	5.221365	1.9361989	5.155006	5.313910
Sample_15	3.888318	1.9679871	1.893506	4.067595
Sample_16	8.573826	4.5383571	8.262674	13.991640
Sample_17	5.995701	2.5728405	3.078720	9
Sample_18	1.066766	1.1689234	7.479895	4618

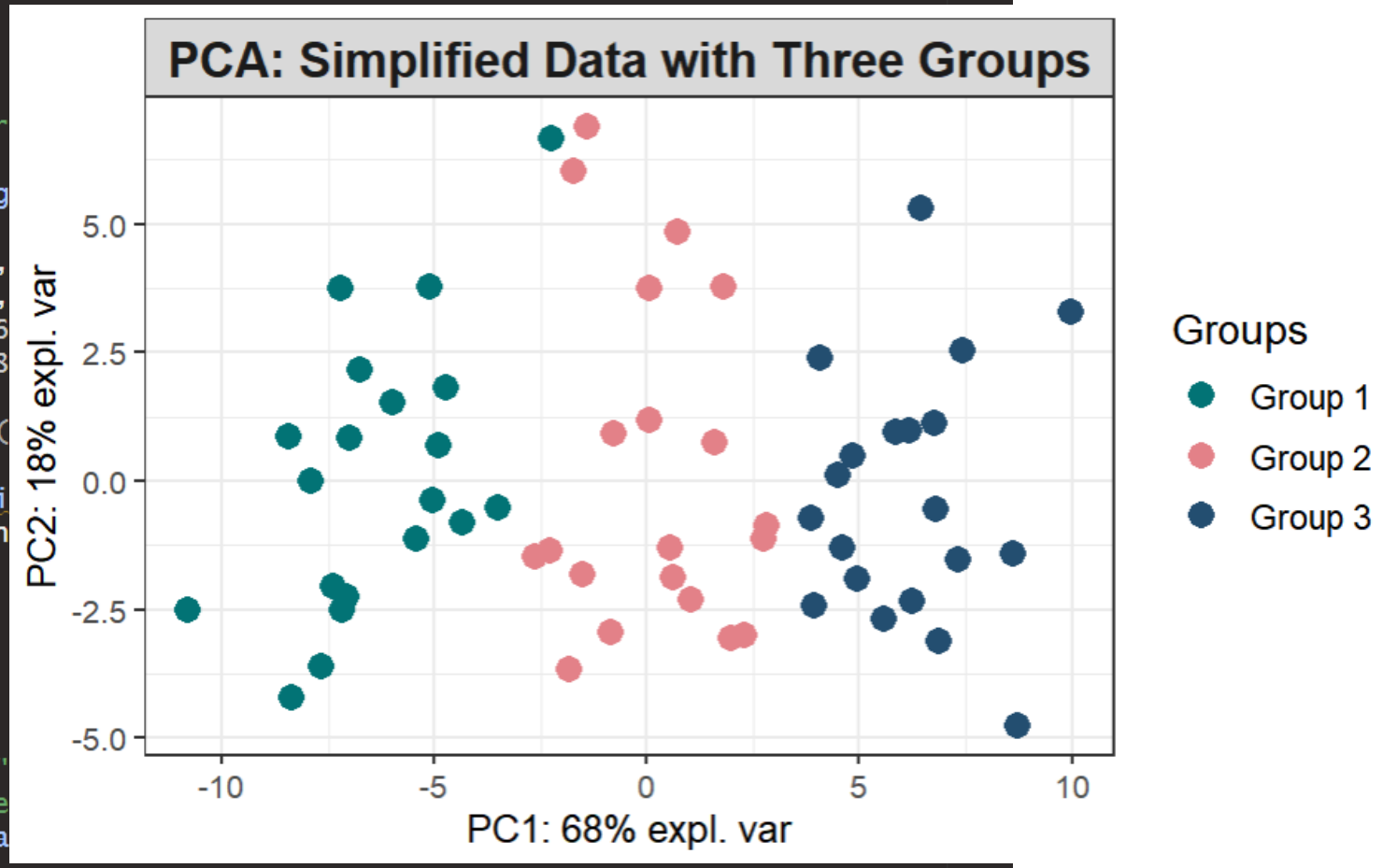
- PCA **simplifies** high-dimensional data by finding a **2 or 3 new dimensions** ("principal components") while **keeping as much of the original variability** as possible
- PCA finds the **best "perspective"** to look at this *n*-dimensional object so that:
  - **Original variance** between the point is **conserved**.
  - **Overlaps** between groups of points are **minimized**.
- **Interpretation**: Points (samples) that are close to each other are similar.

Principal Component Analysis

```r
# Load necessary libraries
library(mixOmics)

# Generate example data
set.seed(123)
group_labels <- rep(c("Group 1", "Group 2", "Group 3"), each = 20)

# Create a dataset with group differences across fewer variables
data <- data.frame(
  Variable1 = c(rnorm(20, 5, 2), rnorm(20, 10, 2), rnorm(20, 15, 2)),
  Variable2 = c(rnorm(20, 3, 1.5), rnorm(20, 7, 1.5), rnorm(20, 11, 1.5)),
  Variable3 = rnorm(60, 6, 2),
  Variable4 = rnorm(60, 8, 3)
)
rownames(data) <- paste0("Sample_", 1:60)

# Perform PCA with mixOmics
pca_result <- pca(data, ncomp = 2)  # Keep the first 2 components for visualization

# Visualize PCA
plotIndiv(
  pca_result,
  group = group_labels,
  legend = TRUE,
  legend.title = "Groups",
  title = "PCA: Simplified Data with Three Groups",
  pch = 16,  # Circle shape for points
  size.title = 14,
  size.subtitle = 12,
  size.legend = 10
)
```

Principal Component Analysis

```r
# Load necessary libraries
library(mixOmics)

# Generate example data
set.seed(123)
group_labels <- rep(c("Gr

# Create a dataset with g
data <- data.frame(
  Variable1 = c(rnorm(20,
  Variable2 = c(rnorm(20,
  Variable3 = rnorm(60, 6
  Variable4 = rnorm(60, 8
)
rownames(data) <- paste0(

# Perform PCA with mixOmi
pca_result <- pca(data, n

# Visualize PCA
plotIndiv(
  pca_result,
  group = group_labels,
  legend = TRUE,
  legend.title = "Groups"
  title = "PCA: Simplifie
  pch = 16,  # Circle sha
  size.title = 14,
  size.subtitle = 12,
  size.legend = 10
)
```



Partial Least Squares Discriminant Analysis

	Variable1	Variable2	Variable3	Variable4
Sample_1	3.879049	3.5694592	6.235293	4.810022
Sample_2	4.539645	2.2465148	4.105051	11.789556
Sample_3	8.117417	2.5001889	5.018885	6.951049
Sample_4	5.141017	1.4721369	5.487816	5.403461
Sample_5	5.258575	1.3923132	9.687724	7.291161
Sample_6	8.430130	3.4552930	4.696100	7.408472
Sample_7	5.921832	3.6723147	6.470773	11.329761
Sample_8	2.469878	3.0795063	6.155922	8.254212
Sample_9	3.626294	4.3834012	4.076287	10.262161
Sample_10	4.108676	6.0751270	5.857384	6.502124
Sample_11	7.448164	2.2634533	8.889102	8.643336
Sample_12	5.719628	-0.4637533	6.903008	7.025942
Sample_13	5.801543	4.5086078	6.082466	8.283751
Sample_14	5.221365	1.9361989	5.155006	5.313910
Sample_15	3.888318	1.9679871	1.893506	4.067595
Sample_16	8.573826	4.5383571	8.262674	13.991640
Sample_17	5.995701	2.5728405	3.078720	9
Sample_18	1.066766	1.1689234	7.479895	4618

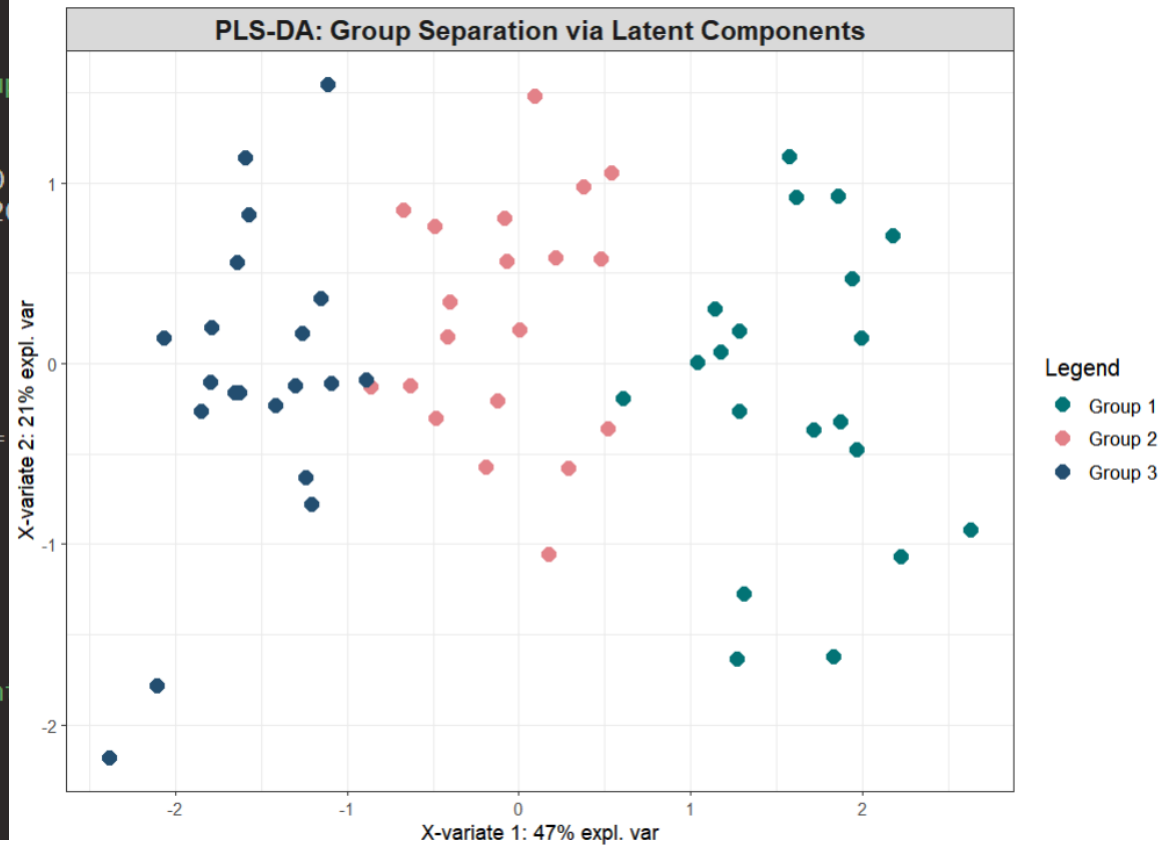
- PCA is **unsupervised** → ignores group labels
- Sometimes we know the classes (e.g. treatment vs control) → want to find components that maximize class separation
- (s)PLS-DA uses class information to orient components toward discrimination (**supervised**)
- **Interpretation**: Points (samples) that are close to each other are similar.

Partial Least Squares Discriminant Analysis

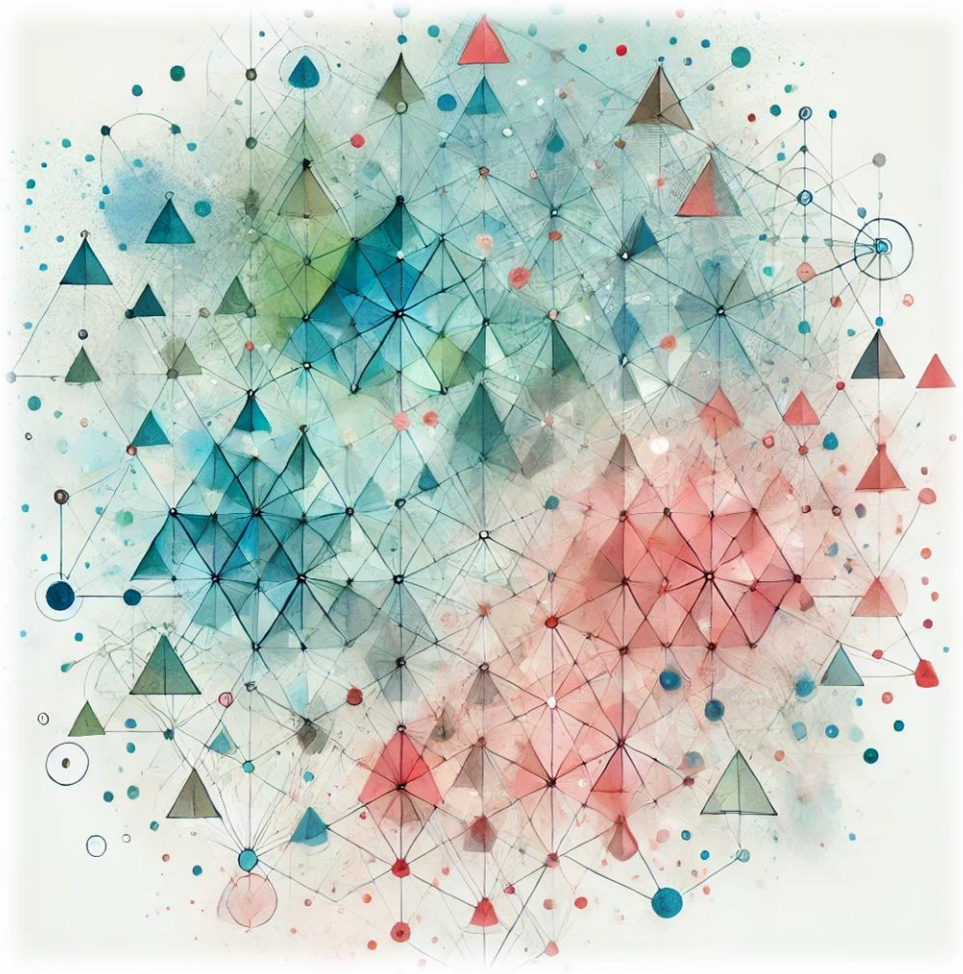
```r
# ──────────────────────────────────────────────
# Example: PLS-DA with three groups
# ──────────────────────────────────────────────
if (!requireNamespace("mixOmics", quietly = TRUE))
  utils::install.packages("mixOmics")

color_palette <- c("#027375", "#e38188", "#234E70")  # Custom colors

set.seed(123)
group_labels <- factor(rep(c("Group 1", "Group 2", "Group 3"), each = 20))

data <- data.frame(
  Var1 = c(rnorm(20, 5, 2),  rnorm(20, 10, 2),  rnorm(20, 15, 2)),
  Var2 = c(rnorm(20, 3, 1.5), rnorm(20, 7, 1.5), rnorm(20, 11, 1.5)),
  Var3 = rnorm(60, 6, 2),
  Var4 = rnorm(60, 8, 3)
)
rownames(data) <- paste0("Sample_", 1:60)

# Perform PLS-DA
plsda_res <- mixOmics::plsda(data, group_labels, ncomp = 2)

# Visualize
mixOmics::plotIndiv(
  plsda_res,
  group  = group_labels,
  col    = color_palette,
  legend = TRUE,
  title  = "PLS-DA: Group Separation via Latent Components",
  pch    = 16,
  size.title = 14,
  size.legend = 10
)
```

Partial Least Squares Discriminant Analysis

```r
# ──────────────────────────────────────────────
# Example: PLS-DA with three groups
# ──────────────────────────────────────────────
if (!requireNamespace("mixOmics", quietly = TRUE))
  utils::install.packages("mixOmics")

color_palette <- c("#027375", "#e38188", "#234E70")  # Custom colors

set.seed(123)
group_labels <- factor(rep(c("Group 1", "Group 2", "Grou

data <- data.frame(
  Var1 = c(rnorm(20, 5, 2),  rnorm(20, 10, 2),  rnorm(20
  Var2 = c(rnorm(20, 3, 1.5), rnorm(20, 7, 1.5), rnorm(2
  Var3 = rnorm(60, 6, 2),
  Var4 = rnorm(60, 8, 3)
)
rownames(data) <- paste0("Sample_", 1:60)

# Perform PLS-DA
plsda_res <- mixOmics::plsda(data, group_labels, ncomp =

# Visualize
mixOmics::plotIndiv(
  plsda_res,
  group   = group_labels,
  col     = color_palette,
  legend  = TRUE,
  title   = "PLS-DA: Group Separation via Latent Componen
  pch     = 16,
  size.title = 14,
  size.legend = 10
)
```



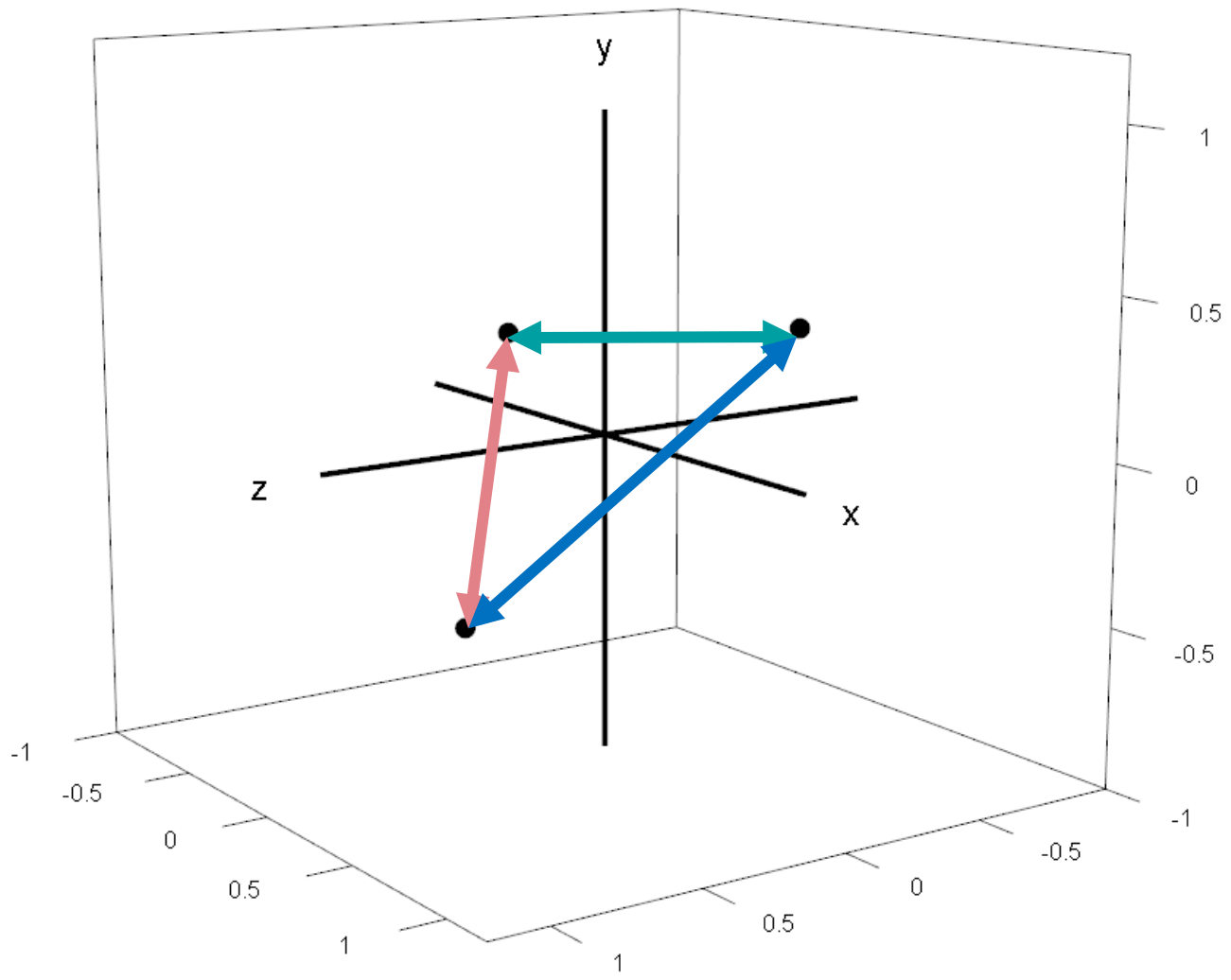
Clustering Algorithms I



- Hierarchical Clustering

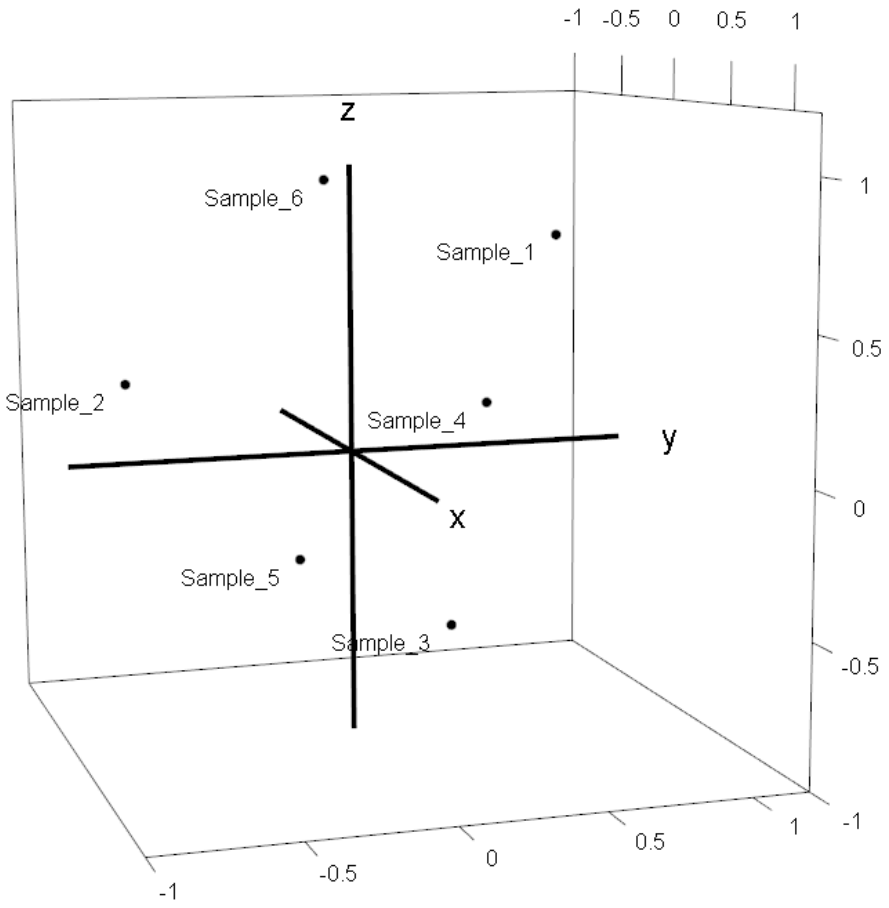
- *k*-Means Clustering

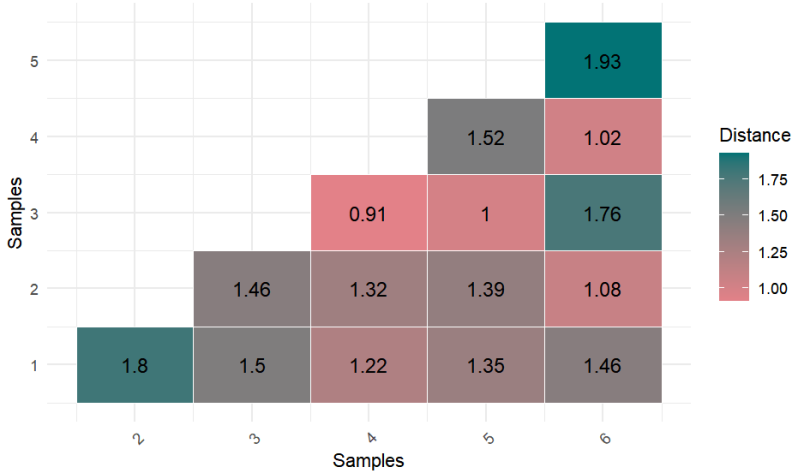
Hierarchical Clustering



- Powerful algorithm to uncover **clusters of points** in an *n*-dimensional space **by their distance**
- **Unsupervised** (no training on example data necessary)
- **Non-parametric** (no knowledge of cluster numbers required)
- **Non-deterministic**, **agglomerative** (cluster-forming)
- Difference to PCA/(s)PLS-DA: does not rely on dimensional reduction → full use of information

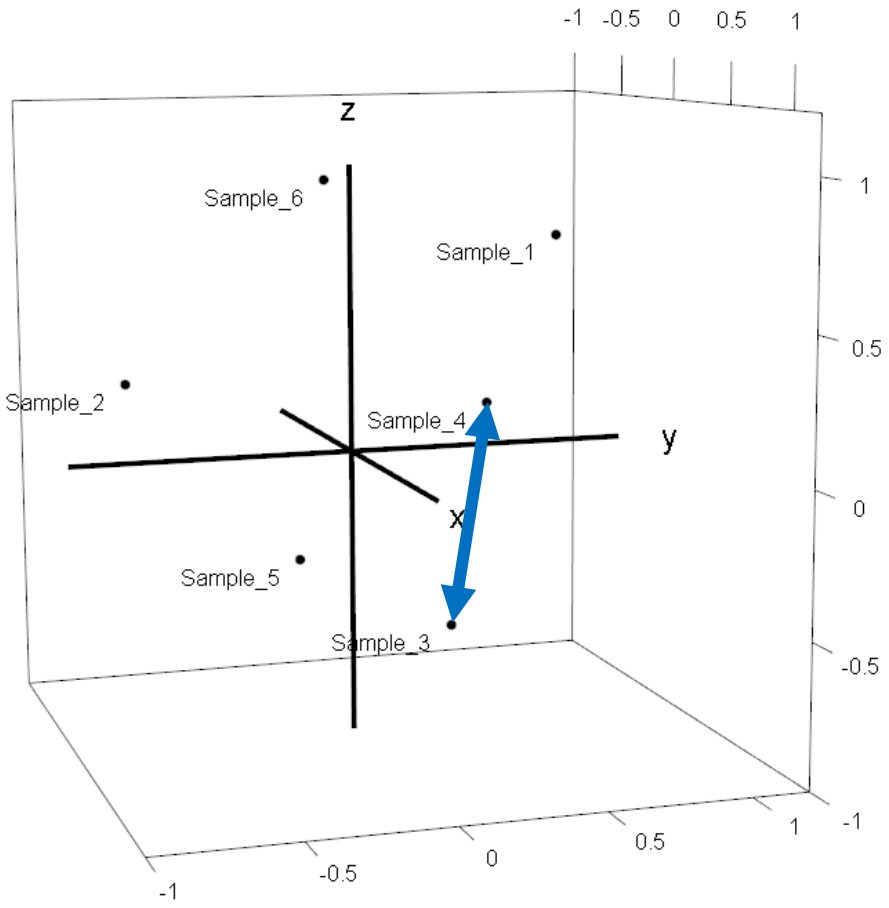
Hierarchical Clustering

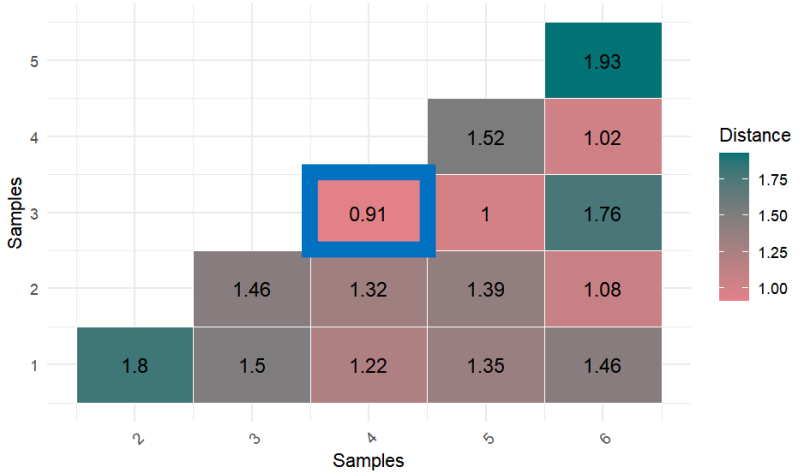




1. Choose the two clusters with the lowest distance

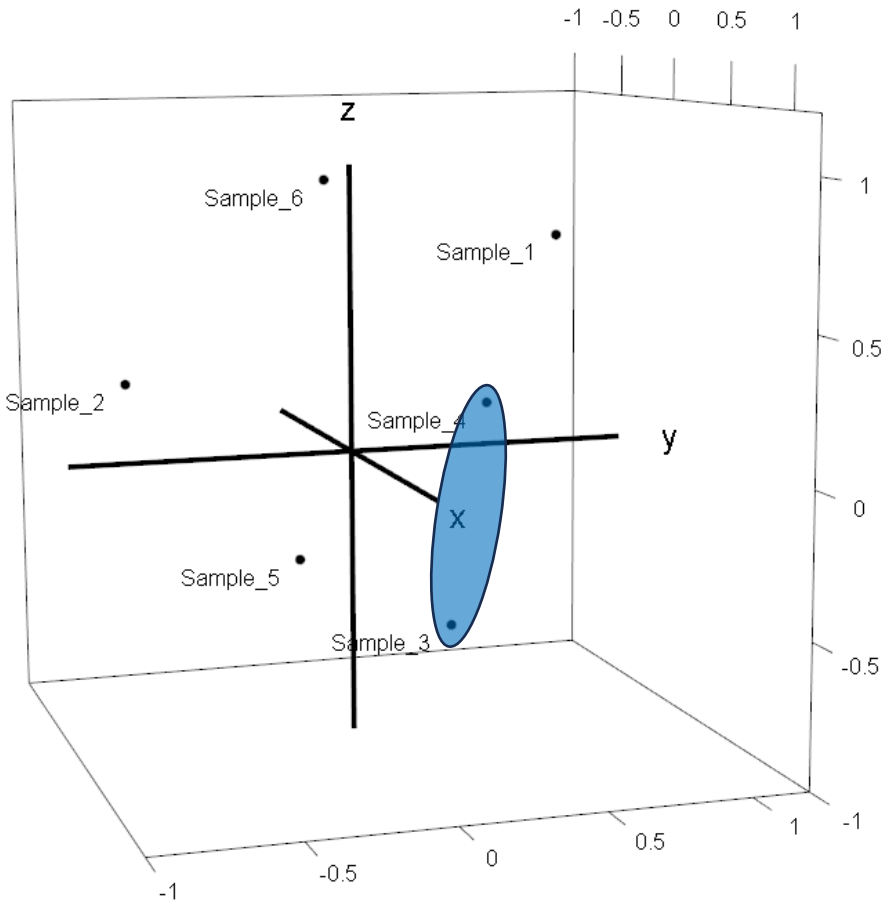
Hierarchical Clustering

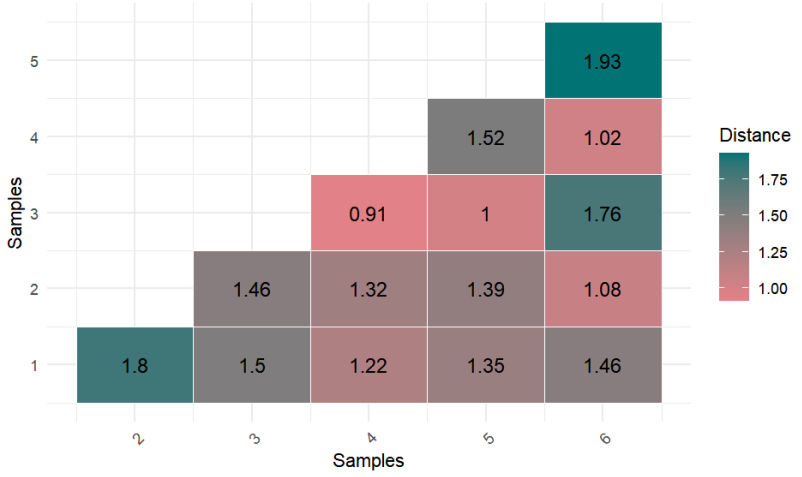




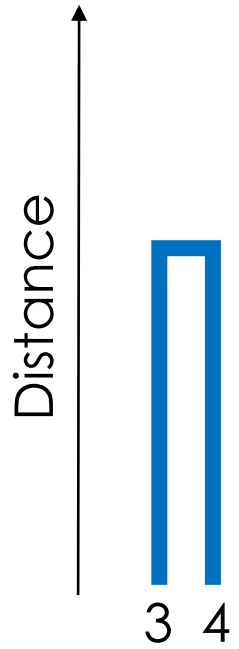
1. Choose the two clusters with the lowest distance

Hierarchical Clustering

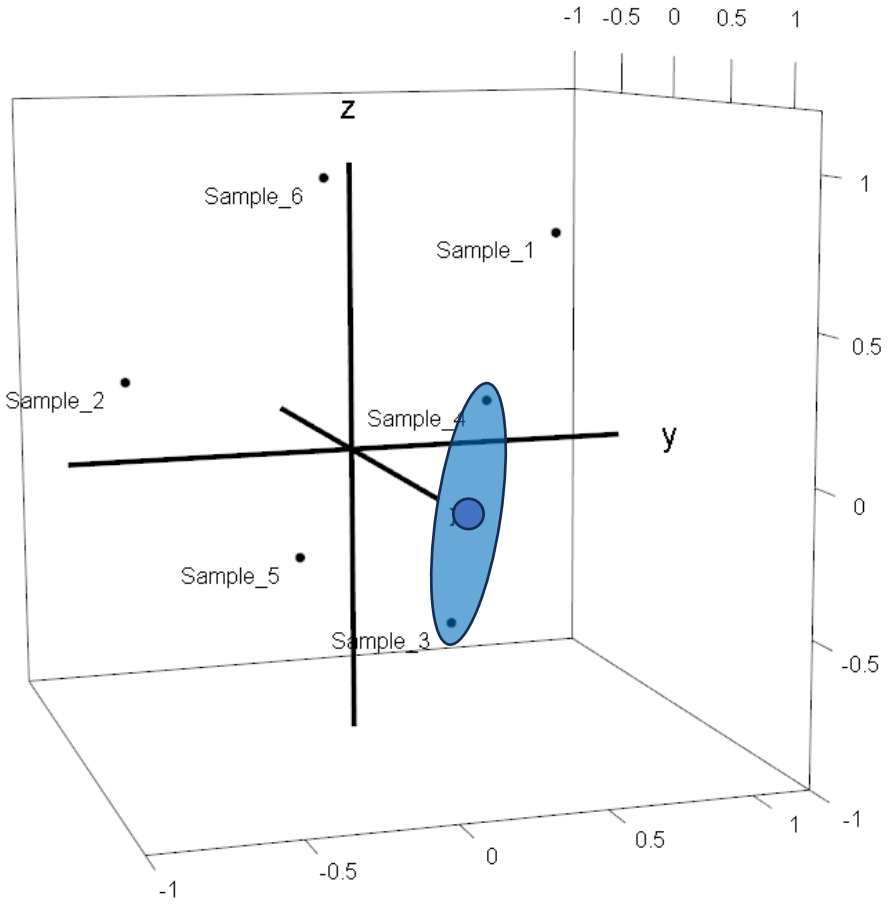


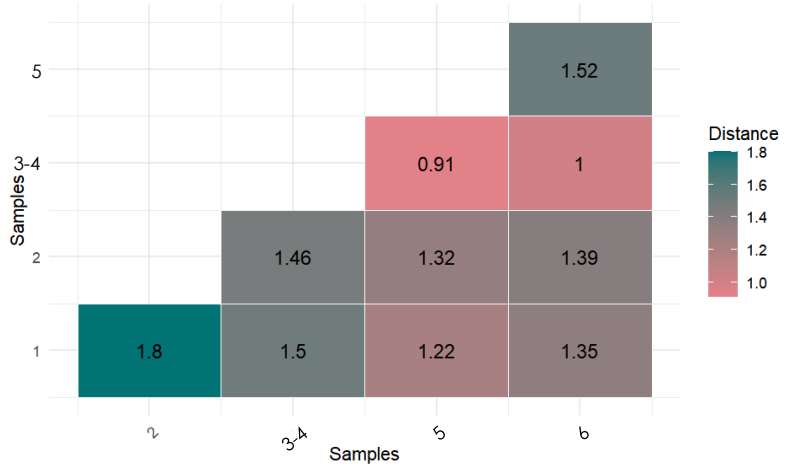


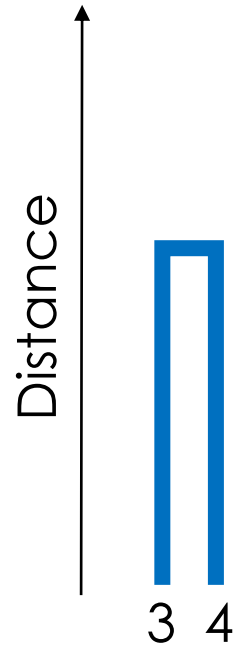
1. Choose the two clusters with the lowest distance
2. Merge them



Hierarchical Clustering

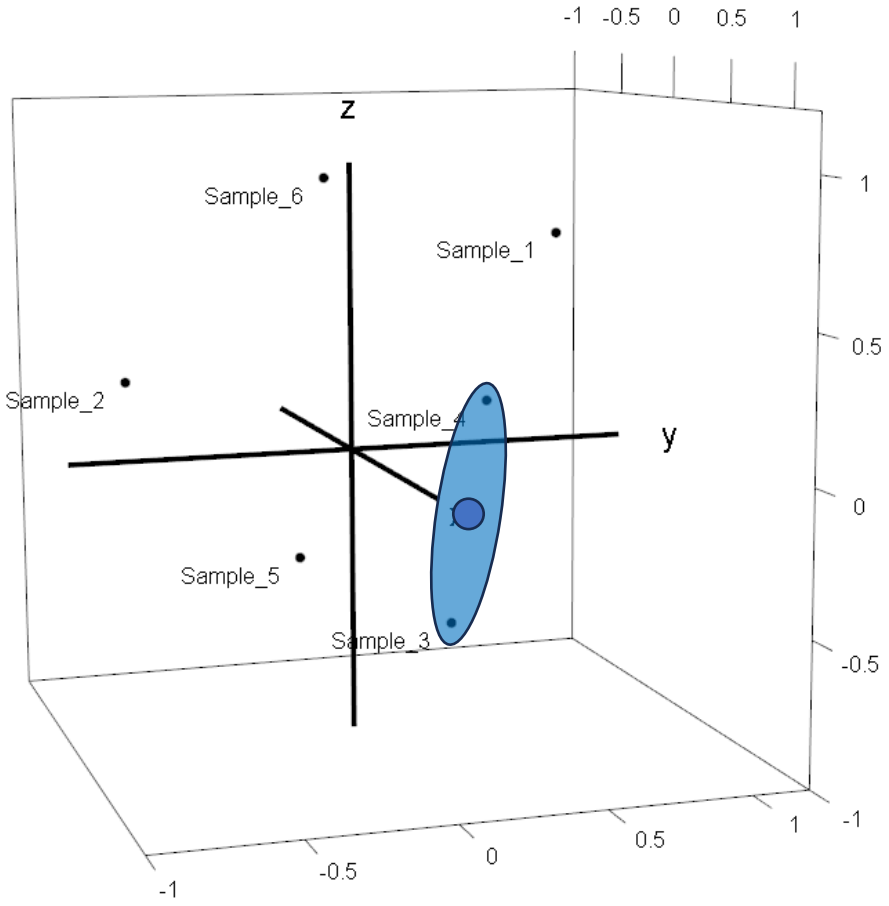


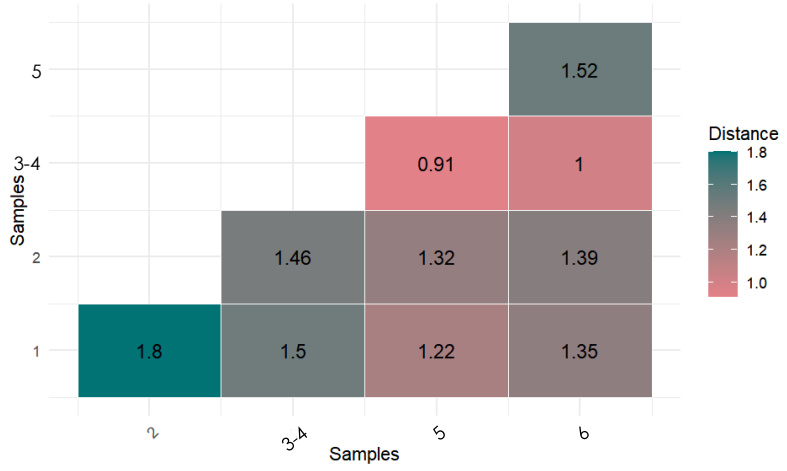


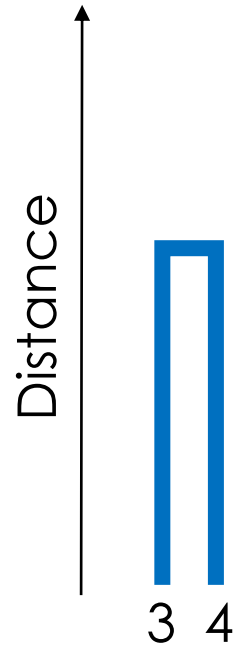


1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4] replacing former samples 3 and 4

Hierarchical Clustering

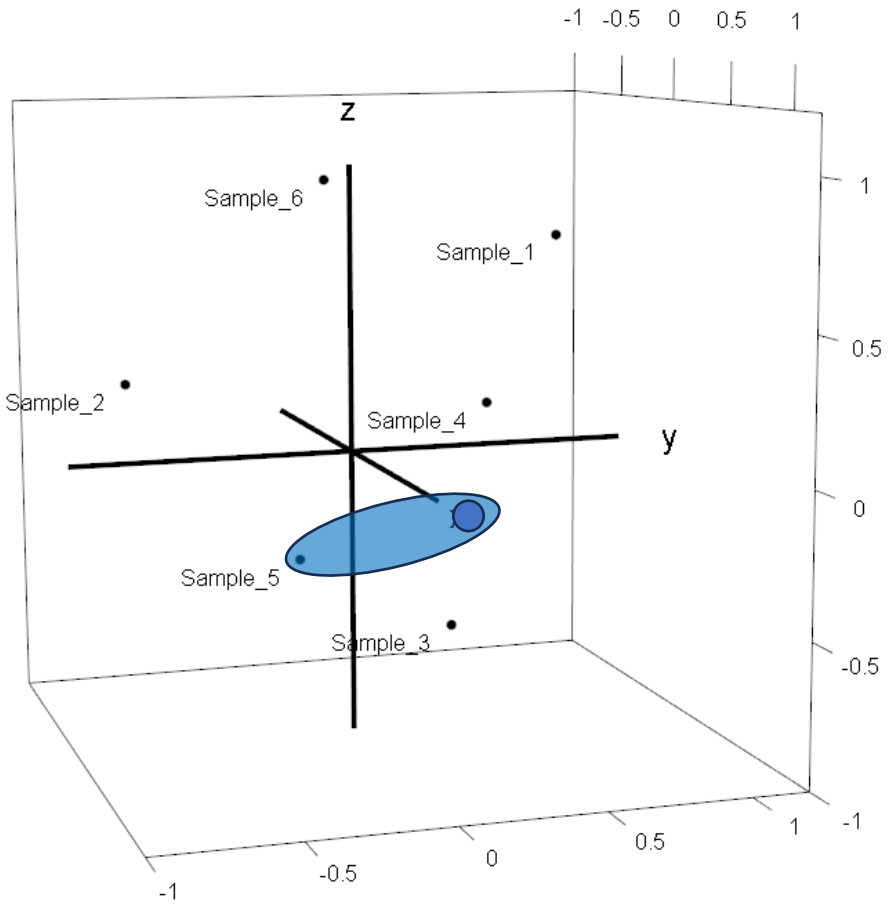


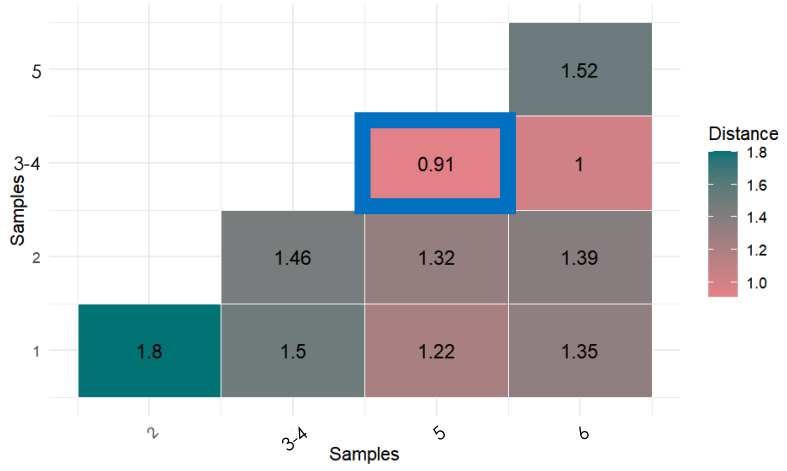


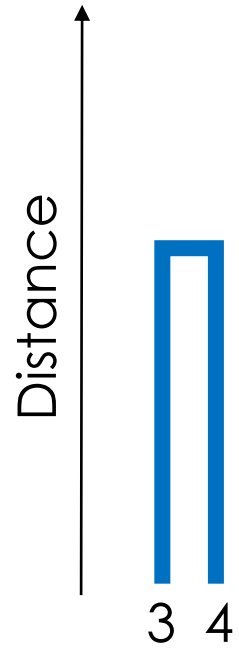


1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

Hierarchical Clustering

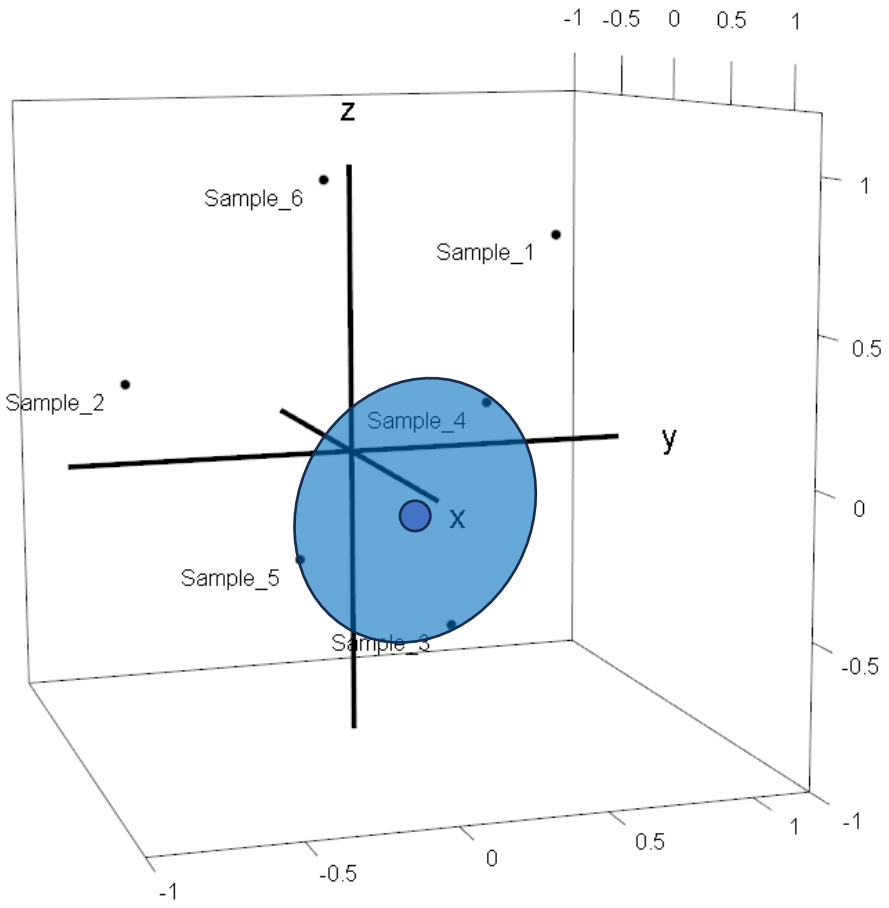


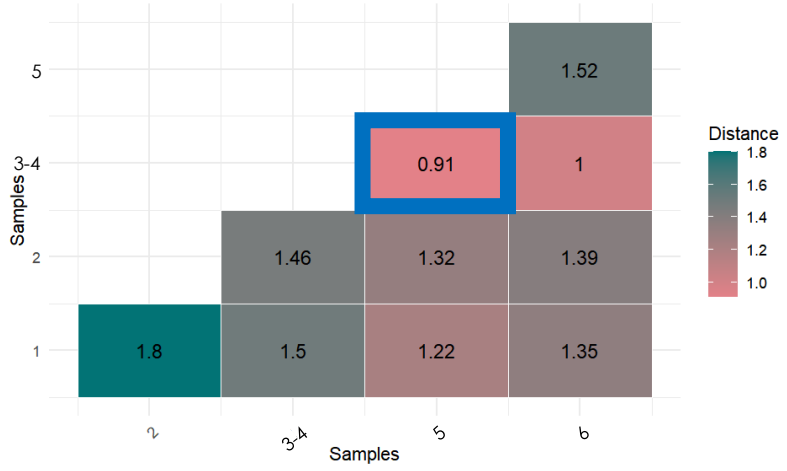


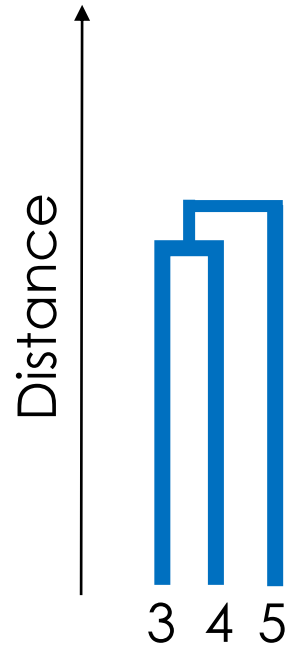


1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

Hierarchical Clustering

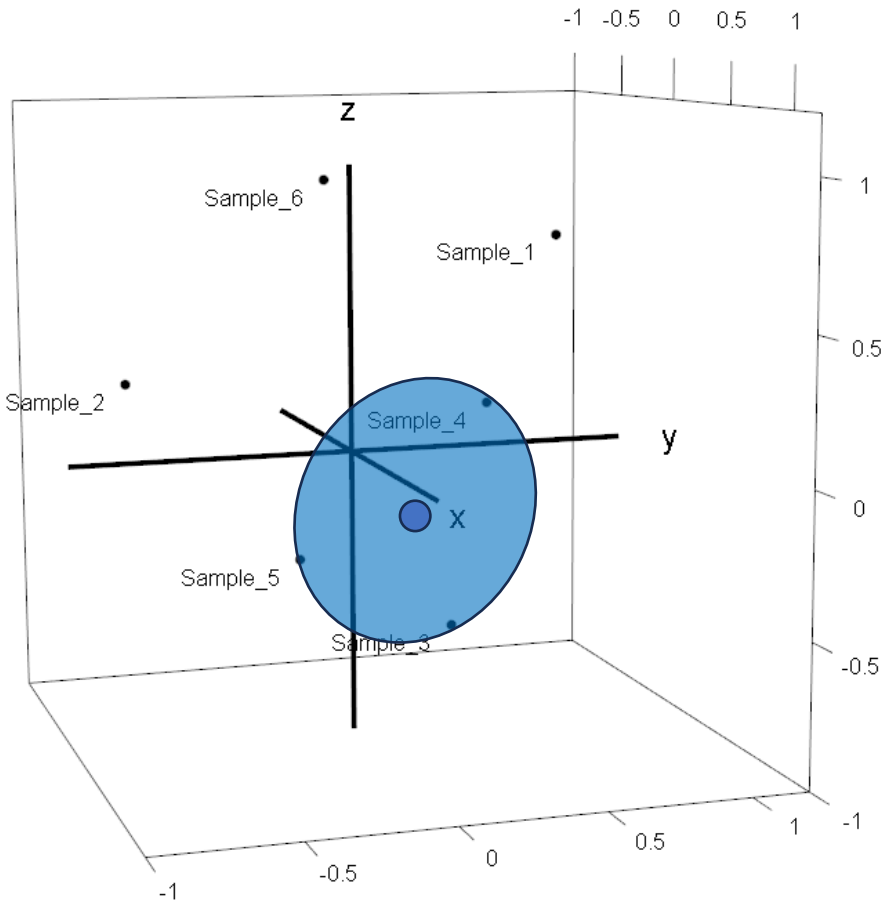






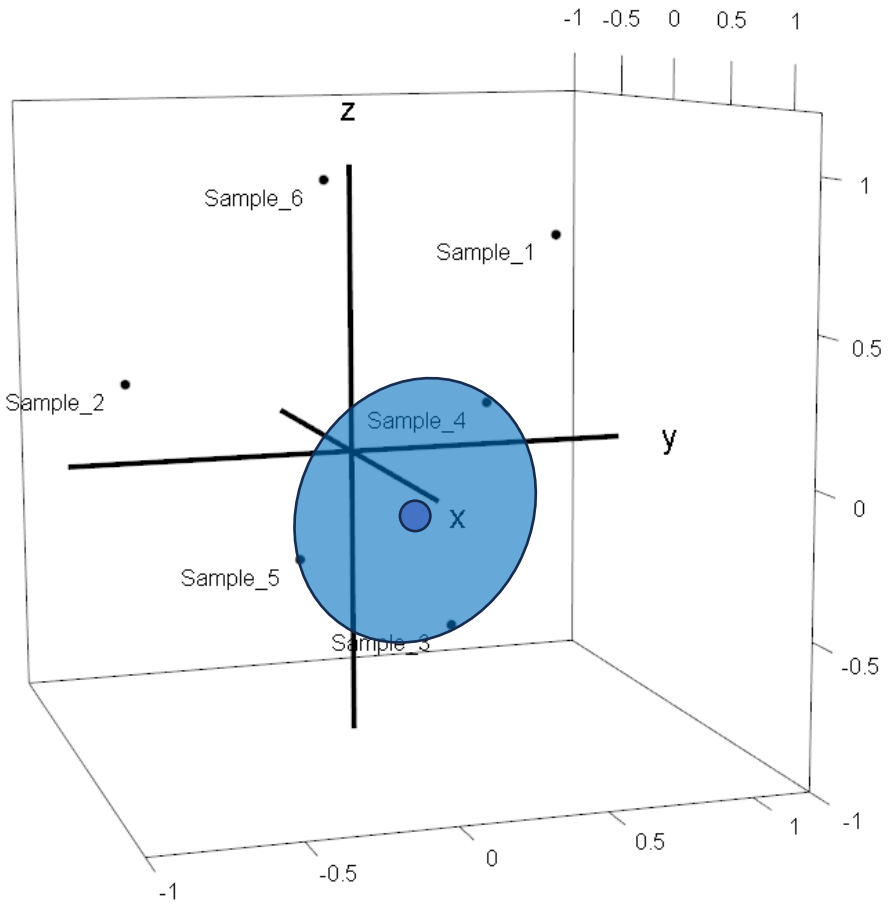
1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

Hierarchical Clustering



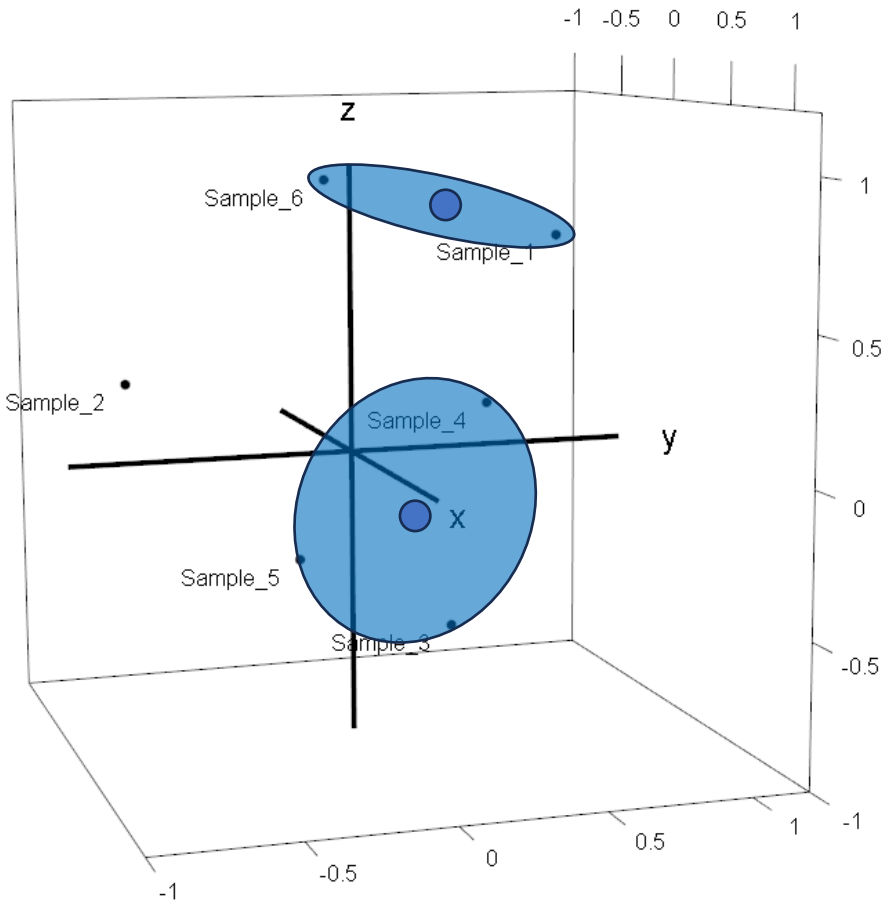
1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

Hierarchical Clustering



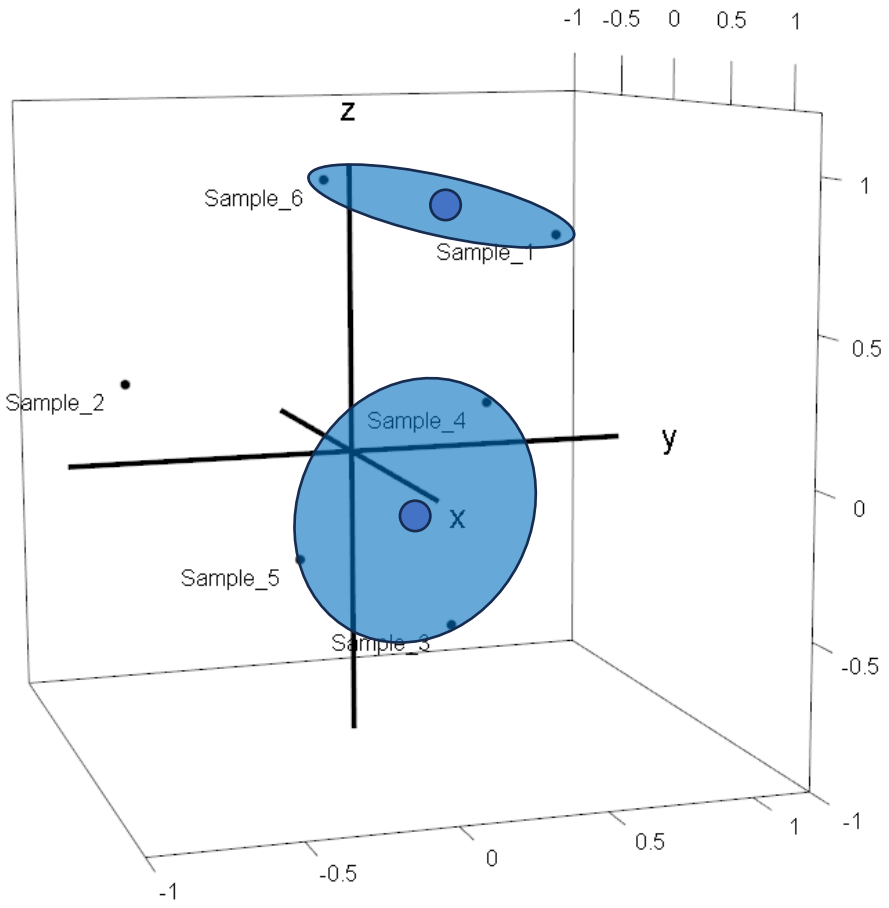
1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

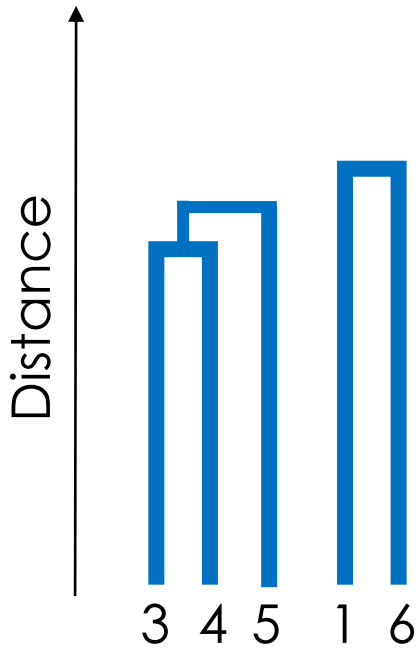
Hierarchical Clustering



1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

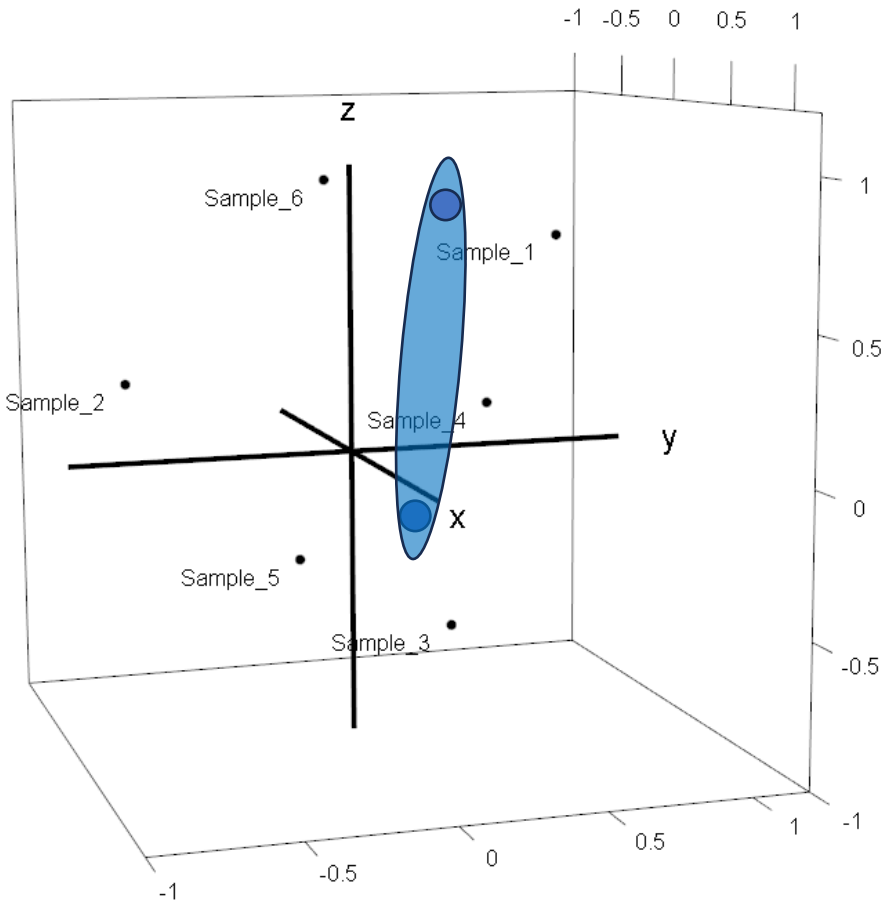
Hierarchical Clustering

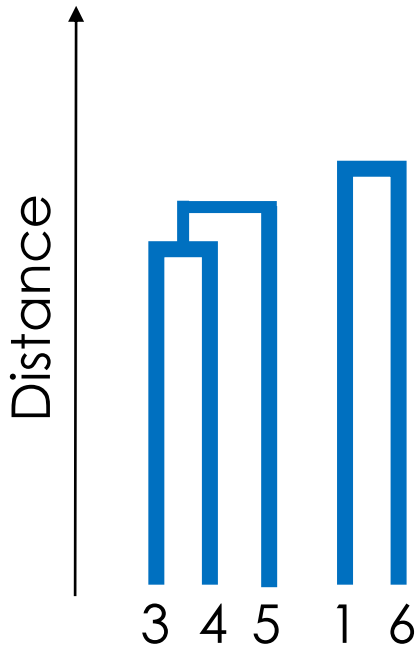




1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

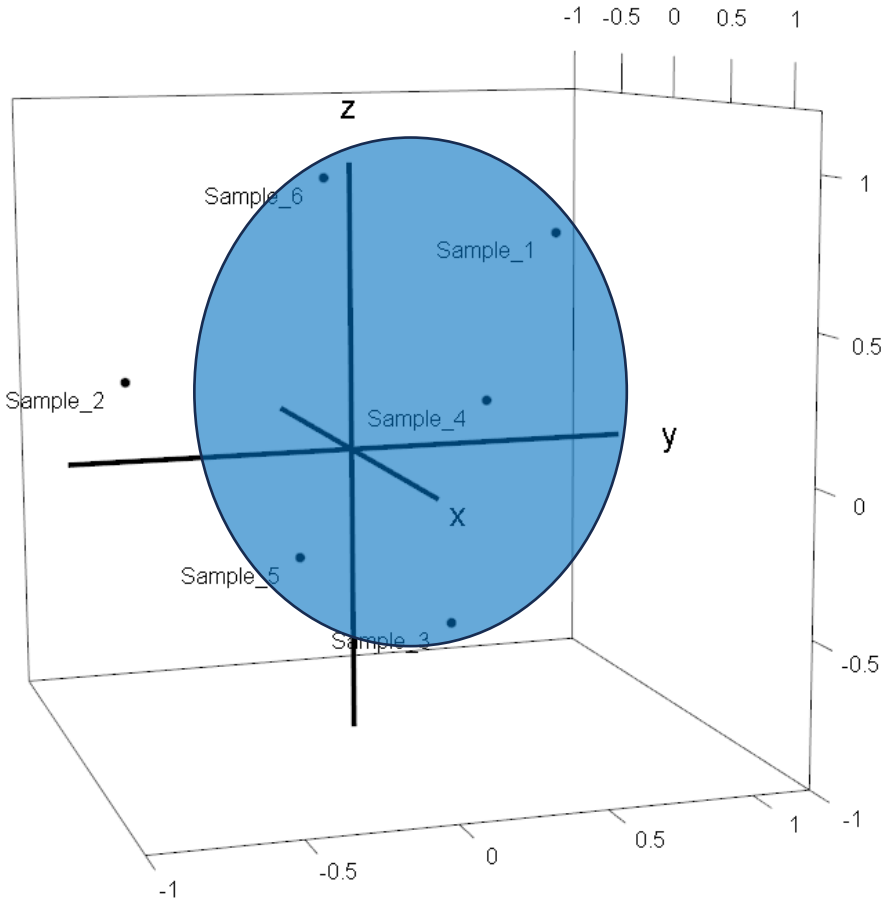
Hierarchical Clustering

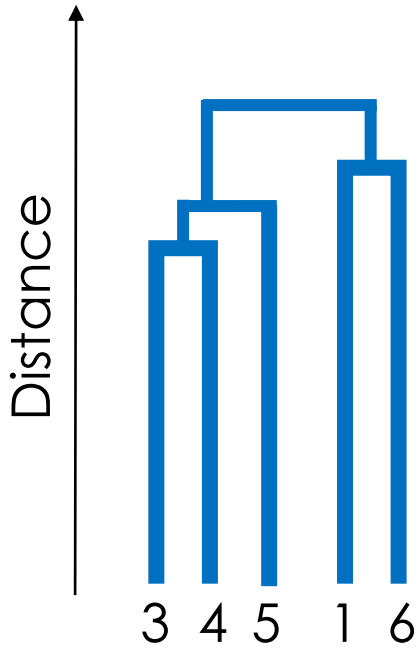




1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

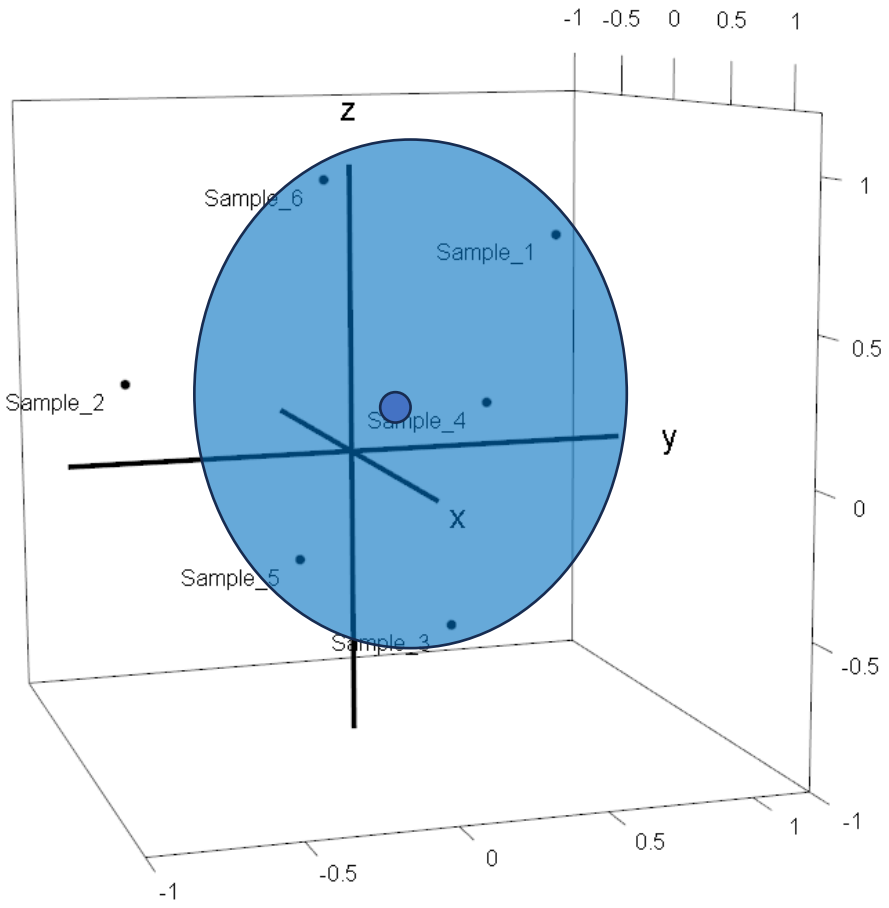
Hierarchical Clustering

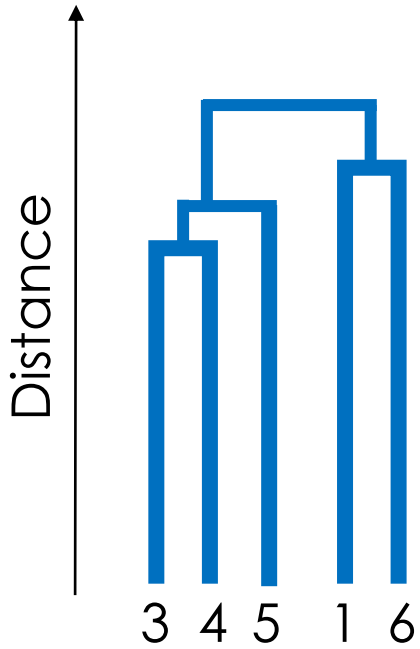




1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

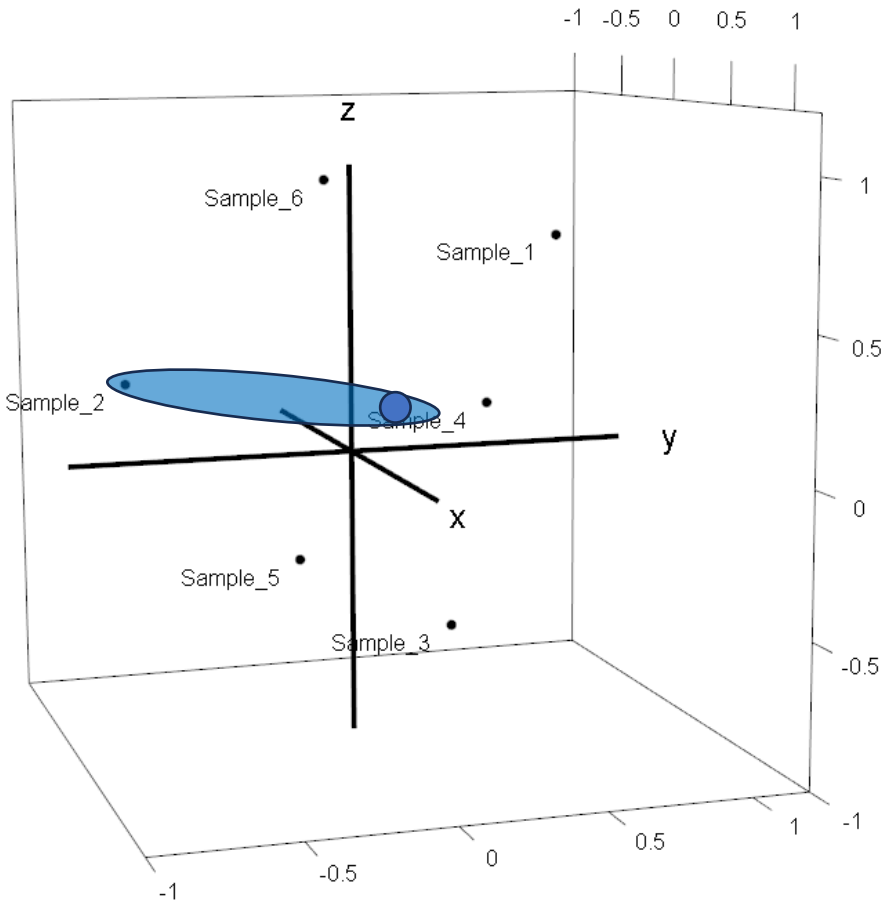
Hierarchical Clustering

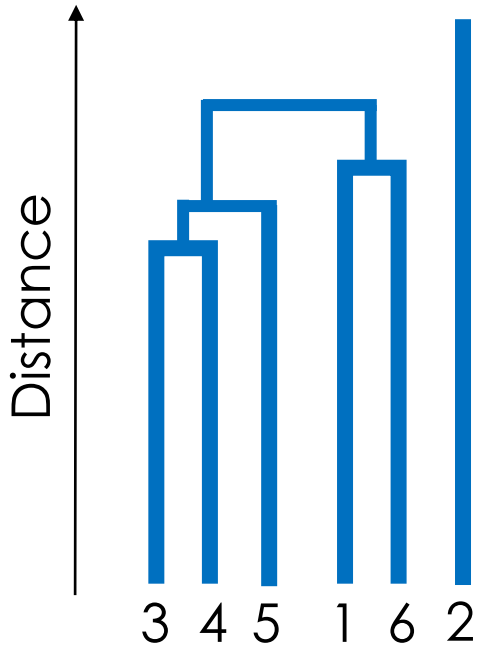




1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

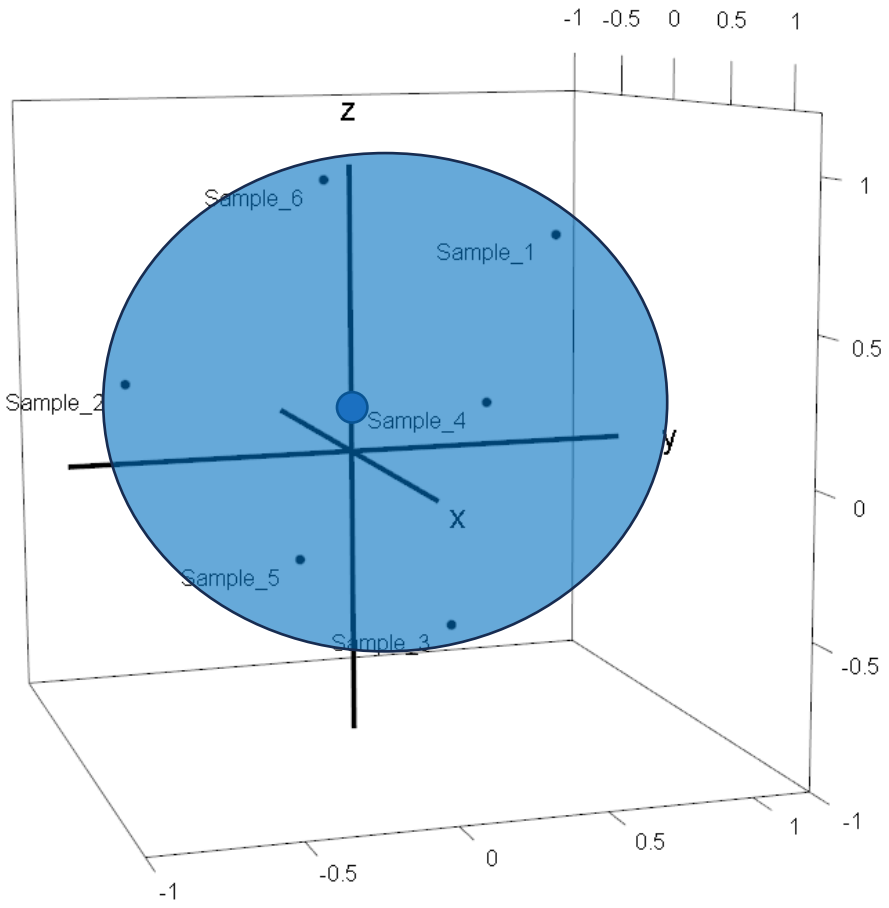
Hierarchical Clustering

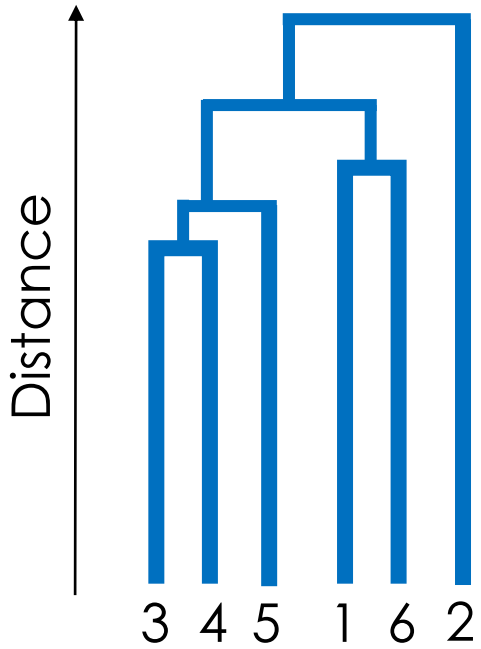




1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

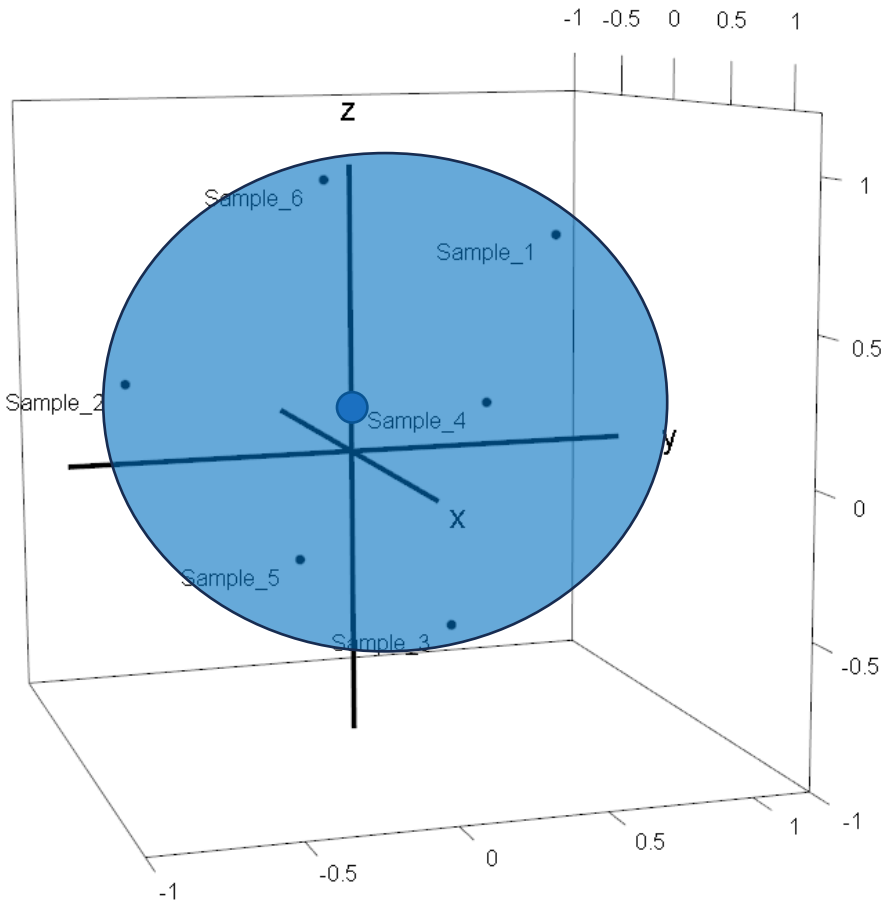
Hierarchical Clustering

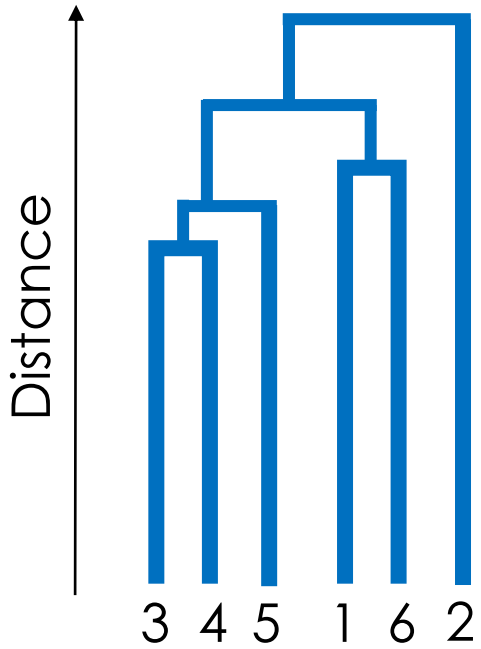




1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.

Hierarchical Clustering





1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.
5. Finish, if no. of clusters = 1

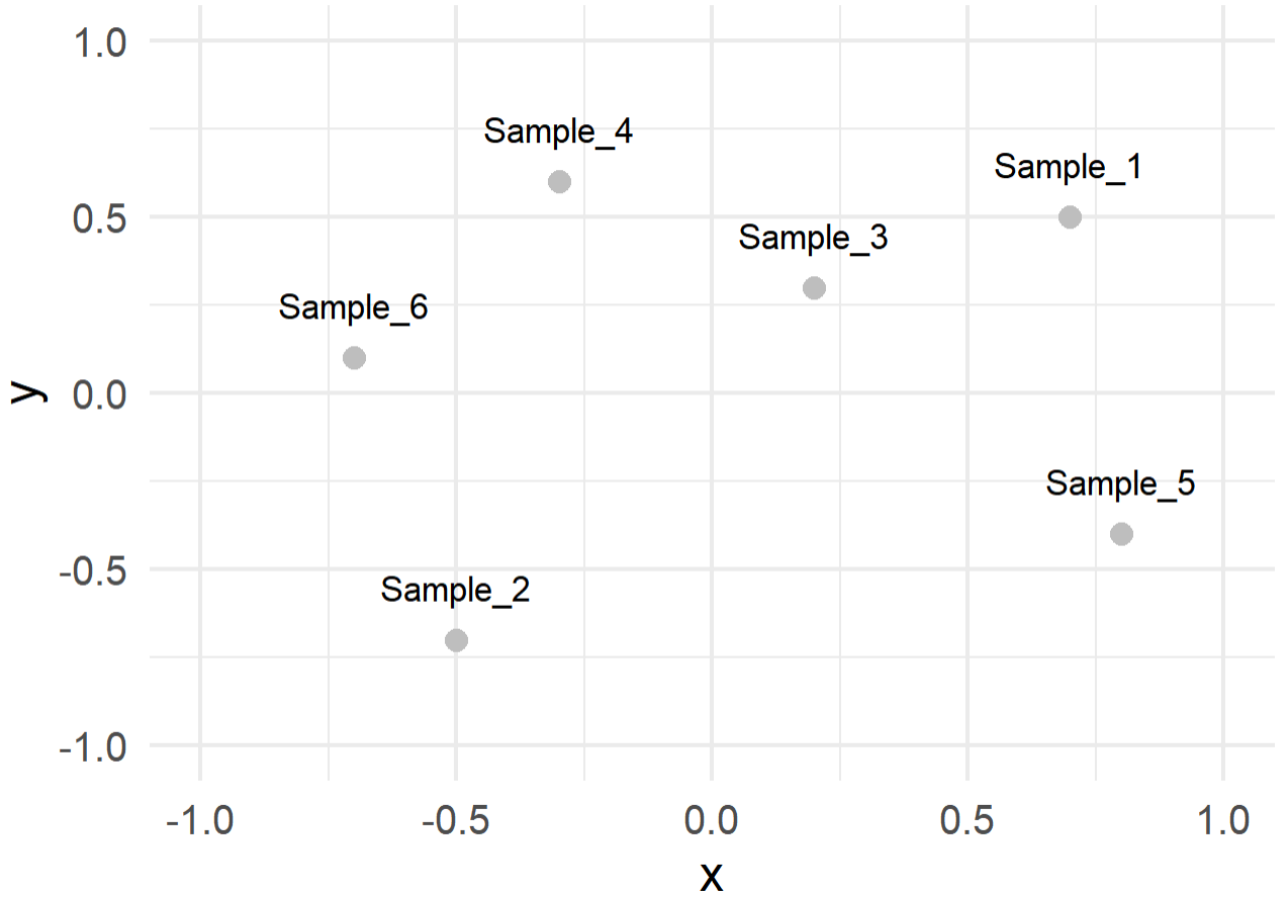
Hierarchical Clustering

```r
# Define six points in 3D space
points_3d <- data.frame(
  Point = paste0("Sample_", 1:6),
  x = c(0.7, -0.5, 0.2, -0.3, 0.8, -0.7),
  y = c(0.5, -0.7, 0.3, 0.6, -0.4, 0.1),
  z = c(0.8, 0.2, -0.6, 0.1, -0.2, 0.9)
)

# Compute the distance matrix
dist_matrix <- dist(points_3d[, c("x", "y", "z")], method = "euclidean")

# Perform hierarchical clustering using the complete linkage method
hclust_result <- hclust(dist_matrix, method = "complete")

# Plot the dendrogram
plot(hclust_result, labels = points_3d$Point,
     main = "Hierarchical Clustering Dendrogram",
     sub = "", xlab = "", ylab = "Distance")
```

1. Choose the two clusters with the lowest distance
2. Merge them
3. Update distance matrix with new [3-4-5] replacing former samples 3-4 and 5
4. Repeat 1.
5. Finish, if no. of clusters = 1

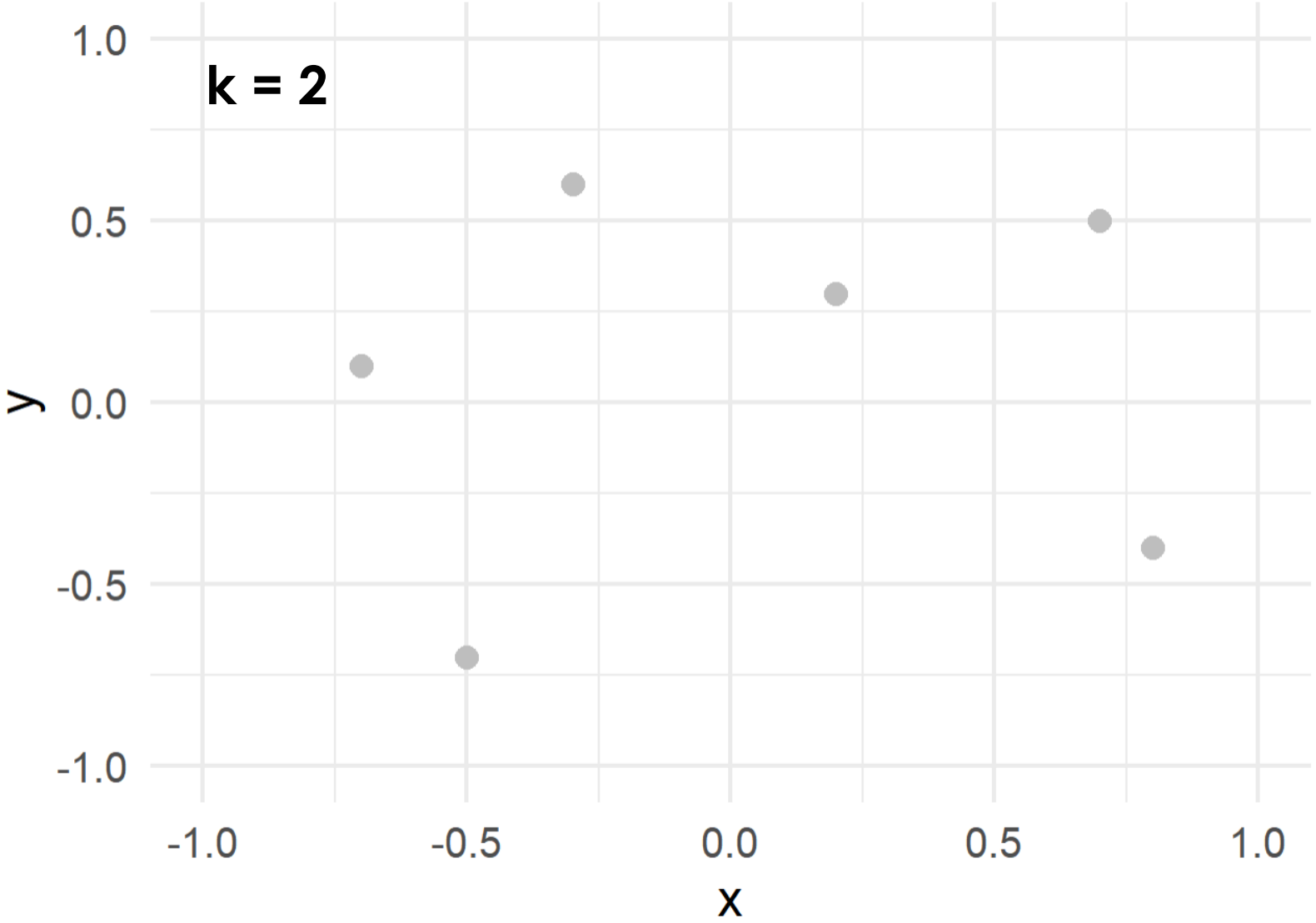
CAVE: Dendrograms may sometimes be hard to interpret. Beware of ambiguity!

k-Means Clustering



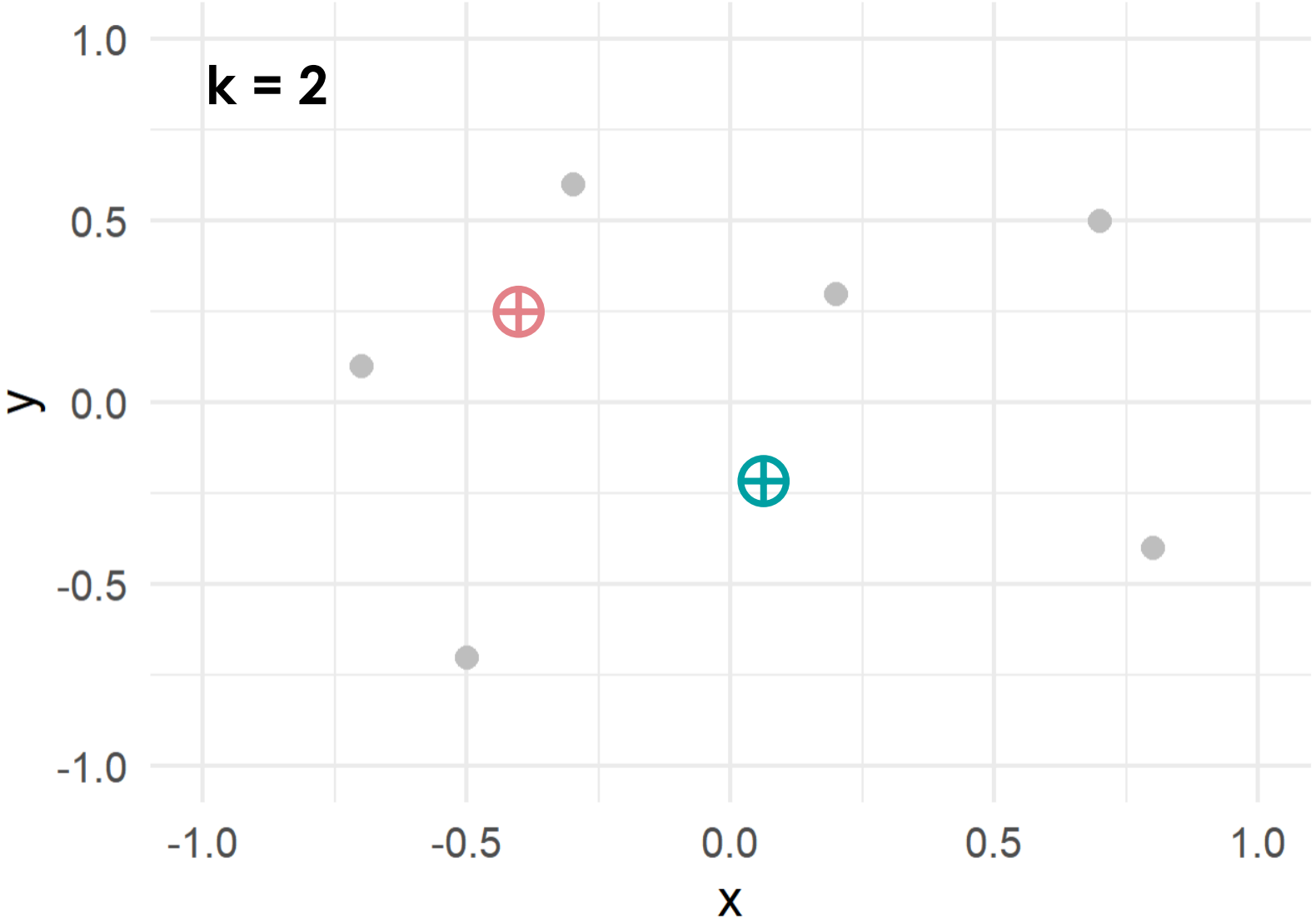
- Powerful algorithm to **assign points** in an *n*-dimensional space **to *k* groups**
- **Unsupervised** (no training on example data necessary)
- **Parametric** (number of clusters must be specified)
- **Deterministic, non-agglomerative** (definite cluster assignment)
- Difference to PCA/(s)PLS-DA: does not rely on dimensional reduction → full use of information

k-Means Clustering



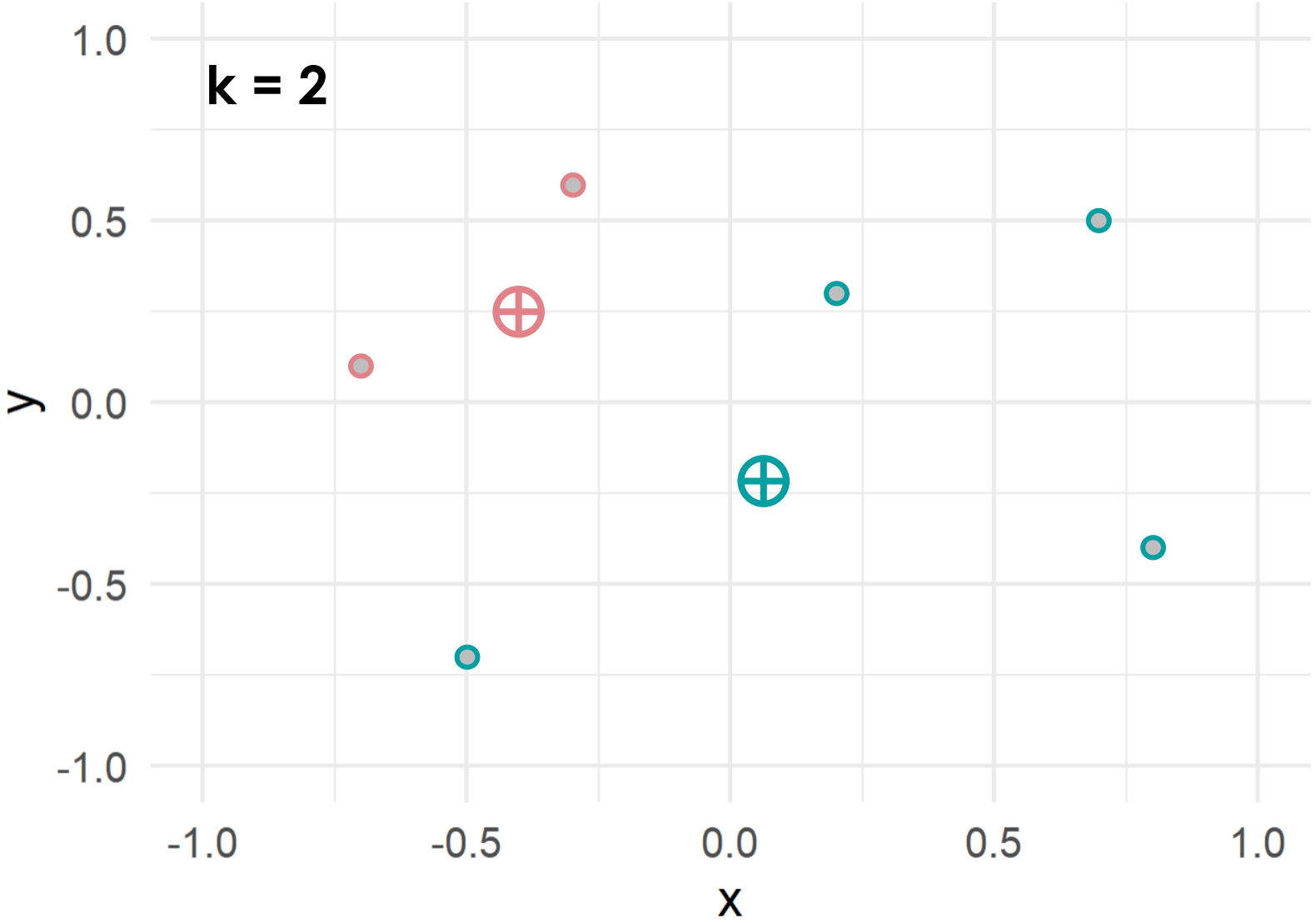
1. Initialize *k* centroids at random positions

k-Means Clustering



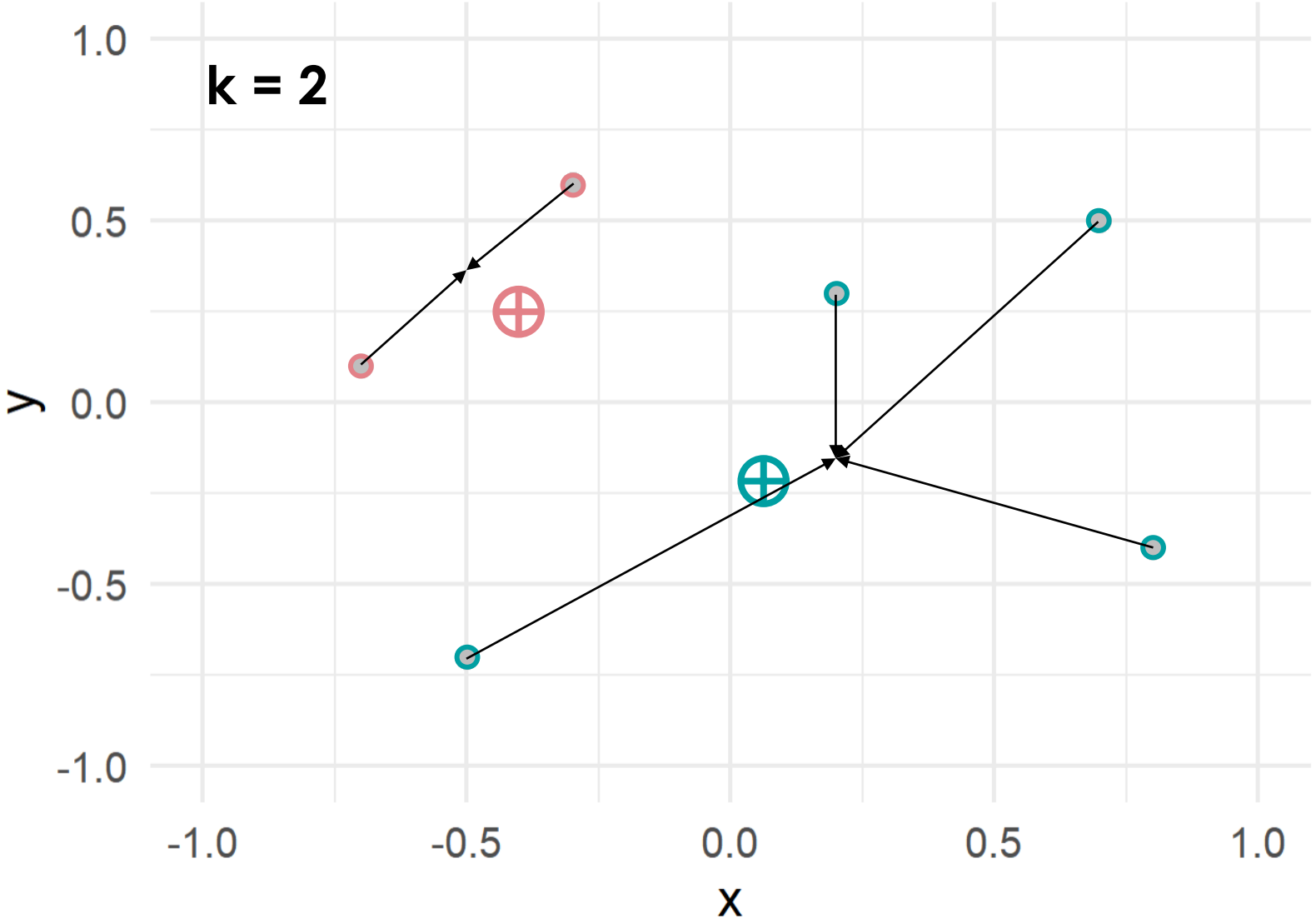
1. Initialize *k* centroids at random positions
2. Assign each sample to the closest centroid

k-Means Clustering



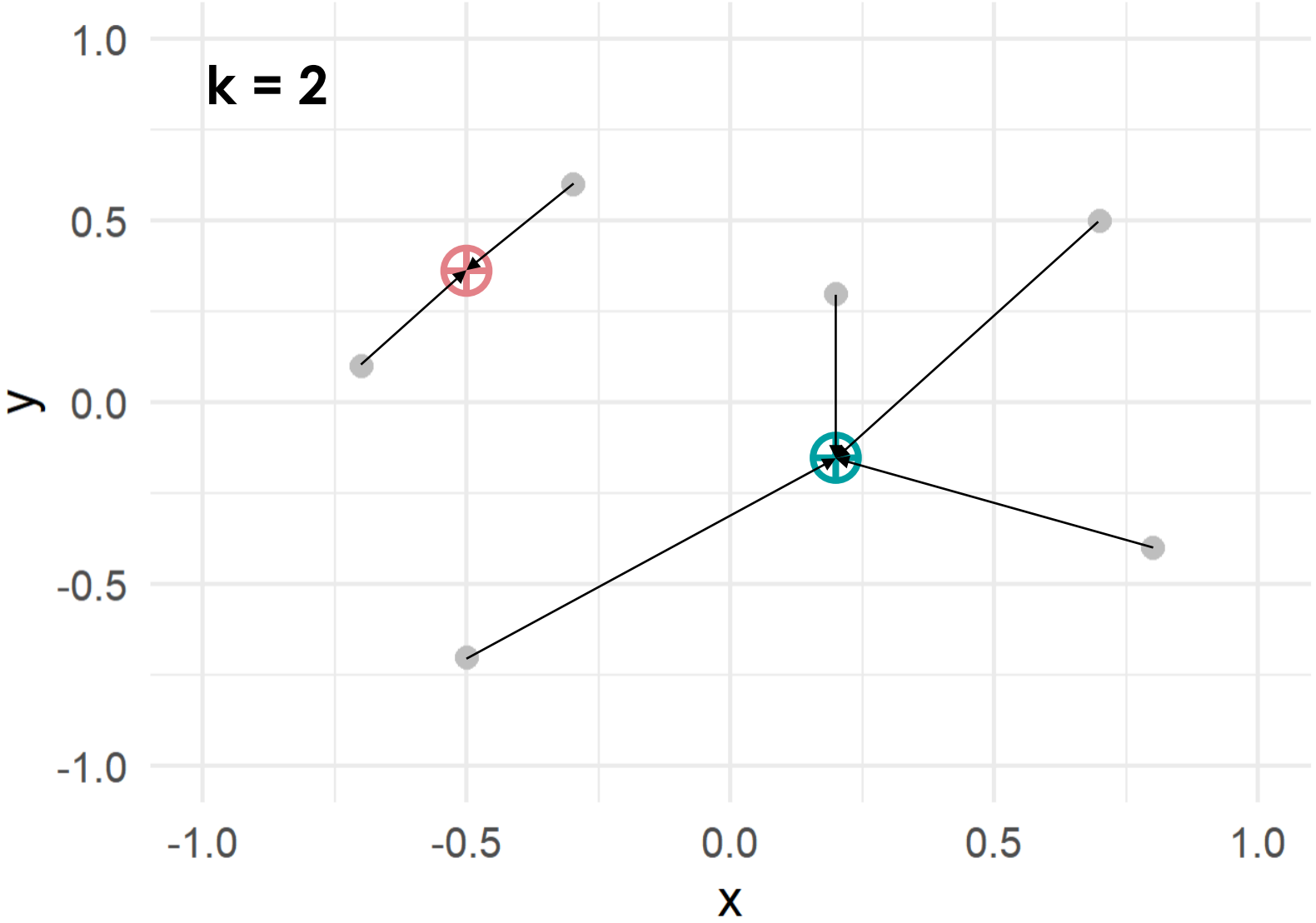
1. Initialize *k* centroids at random positions
2. Assign each sample to the closest centroid
3. Calculate the mean position of the points of each cluster

k-Means Clustering



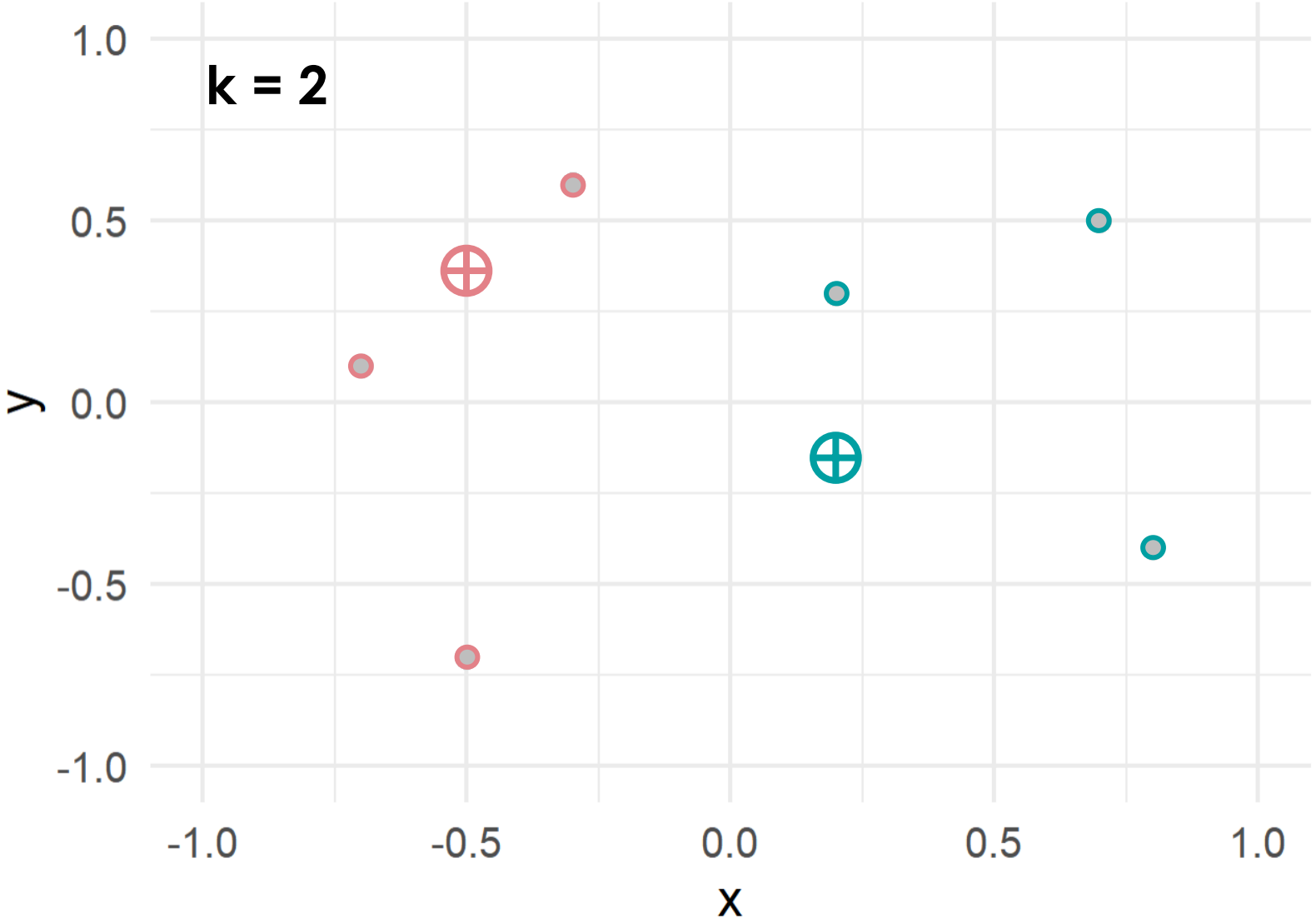
1. Initialize *k* centroids at random positions
2. Assign each sample to the closest centroid
3. Calculate the mean position of the points of each cluster
4. Move the centroids to this position

k-Means Clustering



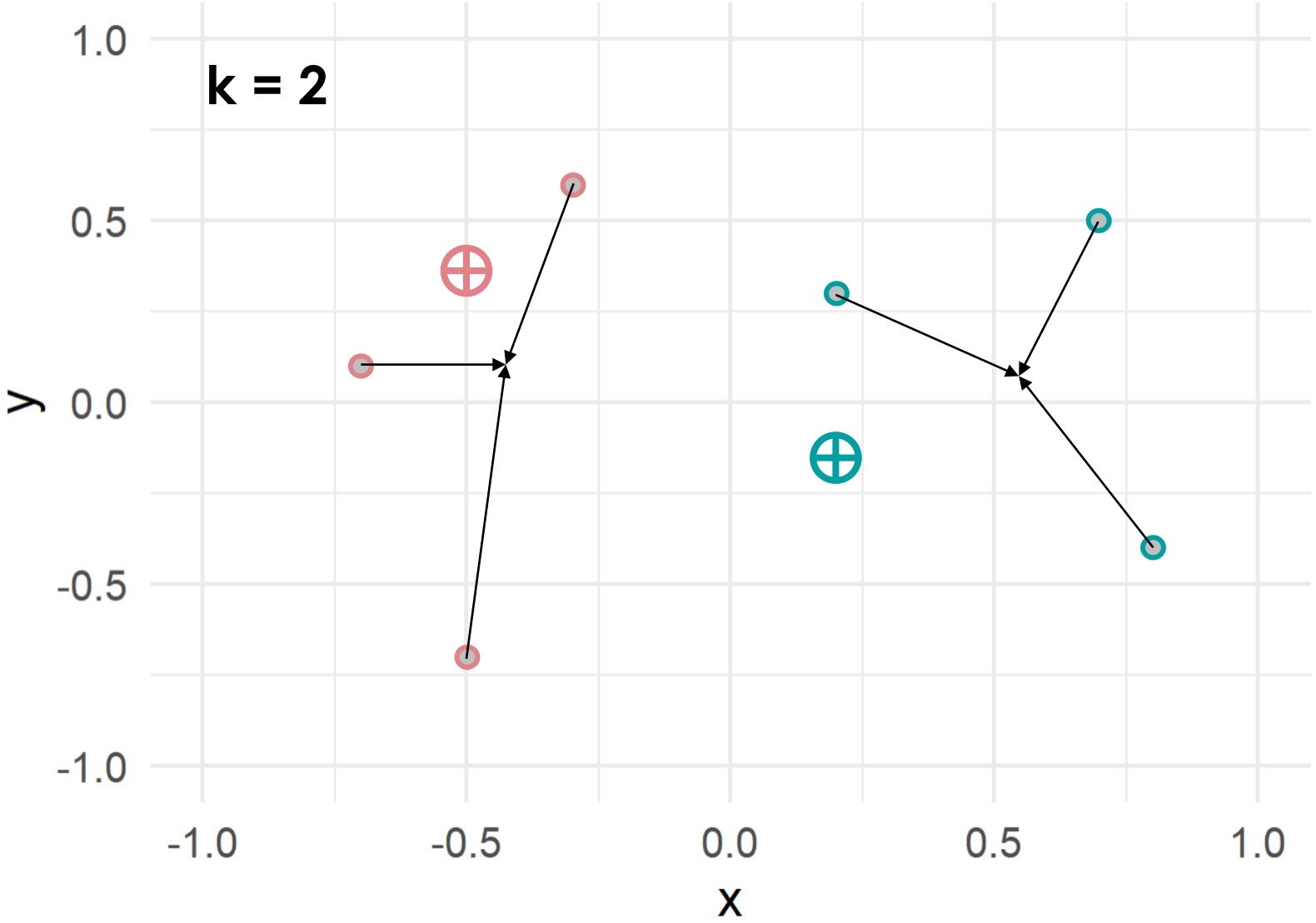
1. Initialize *k* centroids at random positions
2. Assign each sample to the closest centroid
3. Calculate the mean position of the points of each cluster
4. Move the centroids to this position
5. Repeat 2.

k-Means Clustering



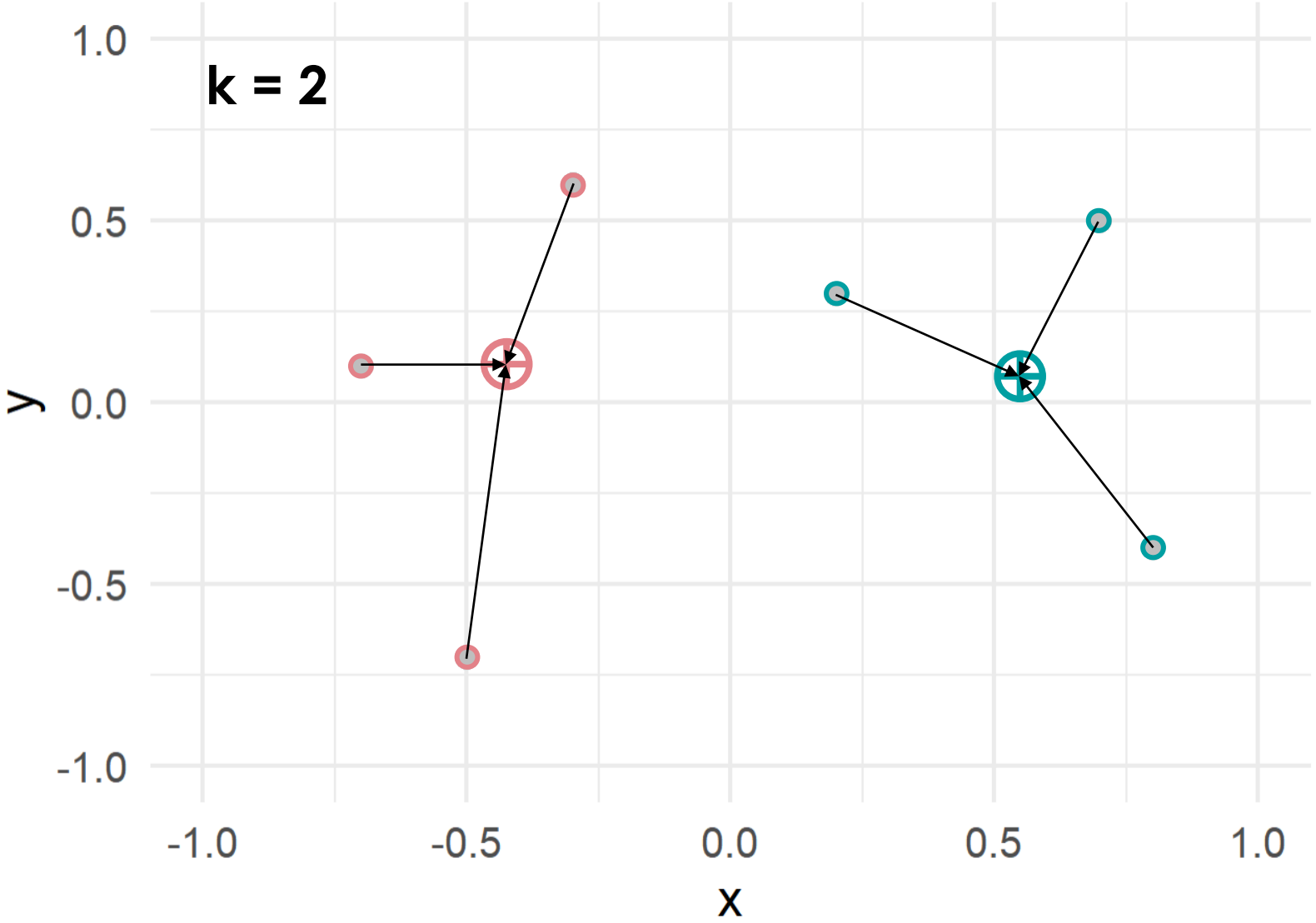
1. Initialize *k* centroids at random positions
2. Assign each sample to the closest centroid
3. Calculate the mean position of the points of each cluster
4. Move the centroids to this position
5. Repeat 2.

k-Means Clustering



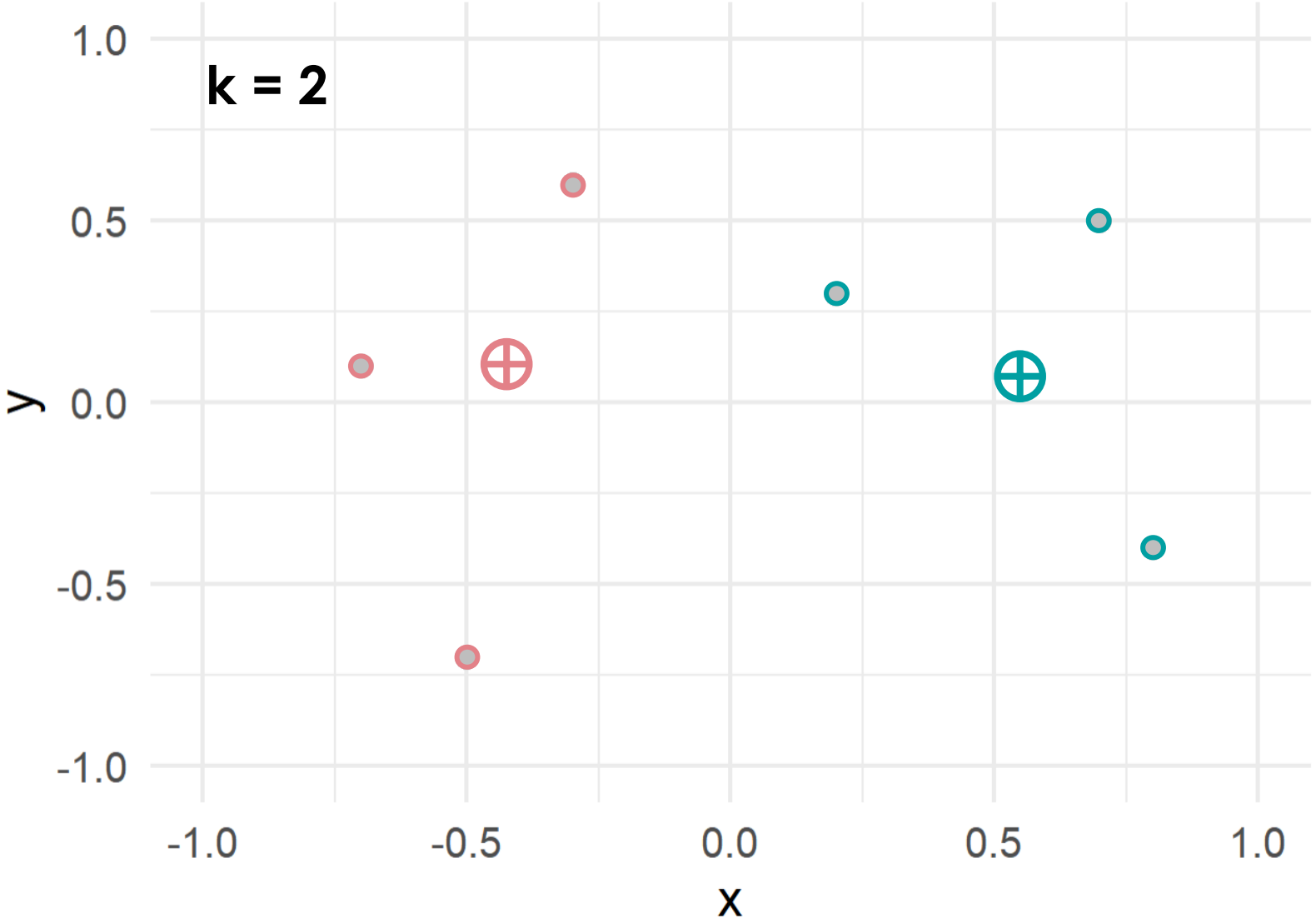
1. Initialize *k* centroids at random positions
2. Assign each sample to the closest centroid
3. Calculate the mean position of the points of each cluster
4. Move the centroids to this position
5. Repeat 2.

k-Means Clustering



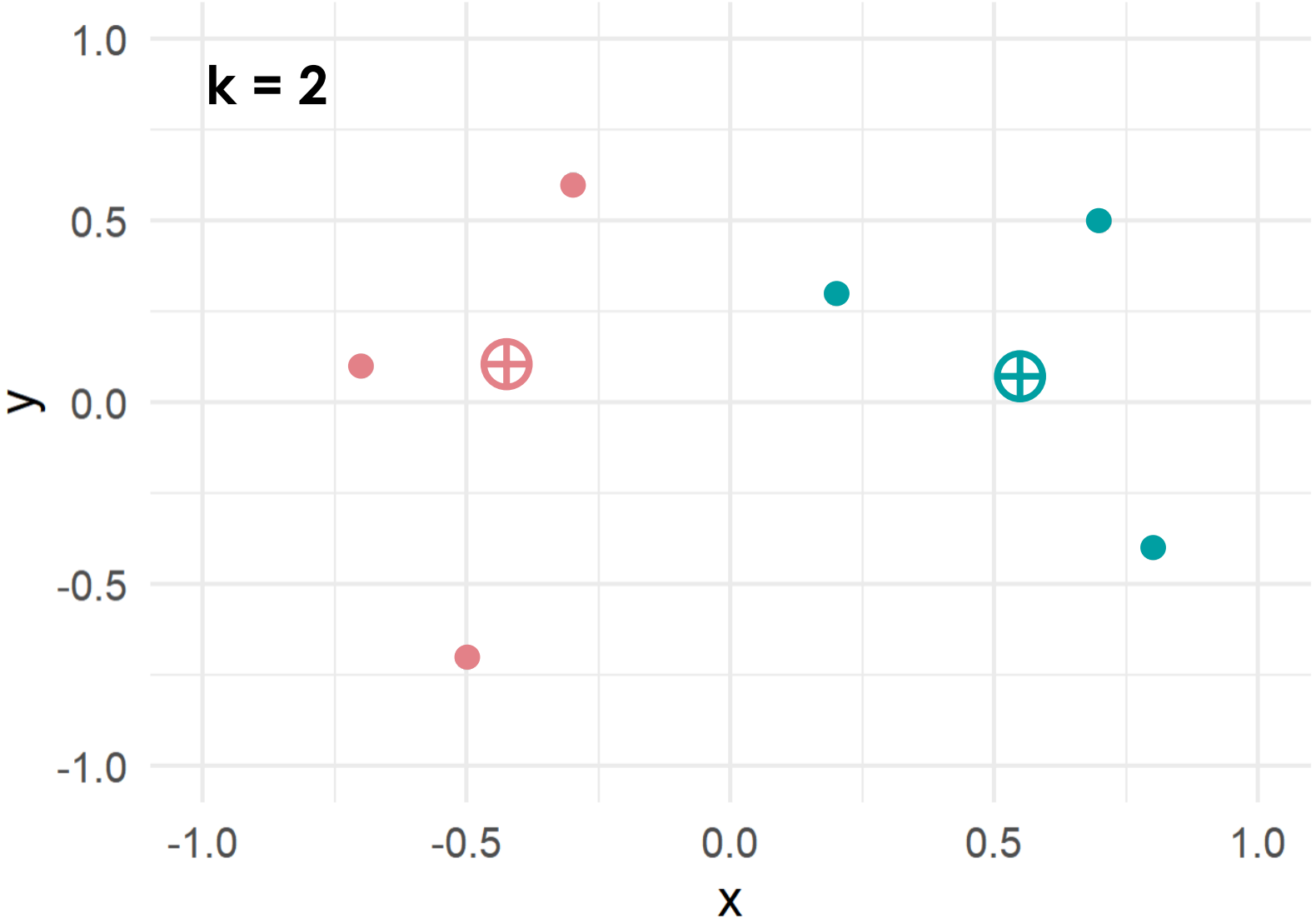
1. Initialize *k* centroids at random positions
2. Assign each sample to the closest centroid
3. Calculate the mean position of the points of each cluster
4. Move the centroids to this position
5. Repeat 2.

k-Means Clustering



1. Initialize *k* centroids at random positions
2. Assign each sample to the closest centroid
3. Calculate the mean position of the points of each cluster
4. Move the centroids to this position
5. Repeat 2.
6. Finish, if:
   a.) Convergence
   b.) Max. iteration number

k-Means Clustering



1. Initialize *k* centroids at random positions
2. Assign each sample to the closest centroid
3. Calculate the mean position of the points of each cluster
4. Move the centroids to this position
5. Repeat 2.
6. Finish, if:
   a.) Convergence
   b.) Max. iteration number

k-Means Clustering

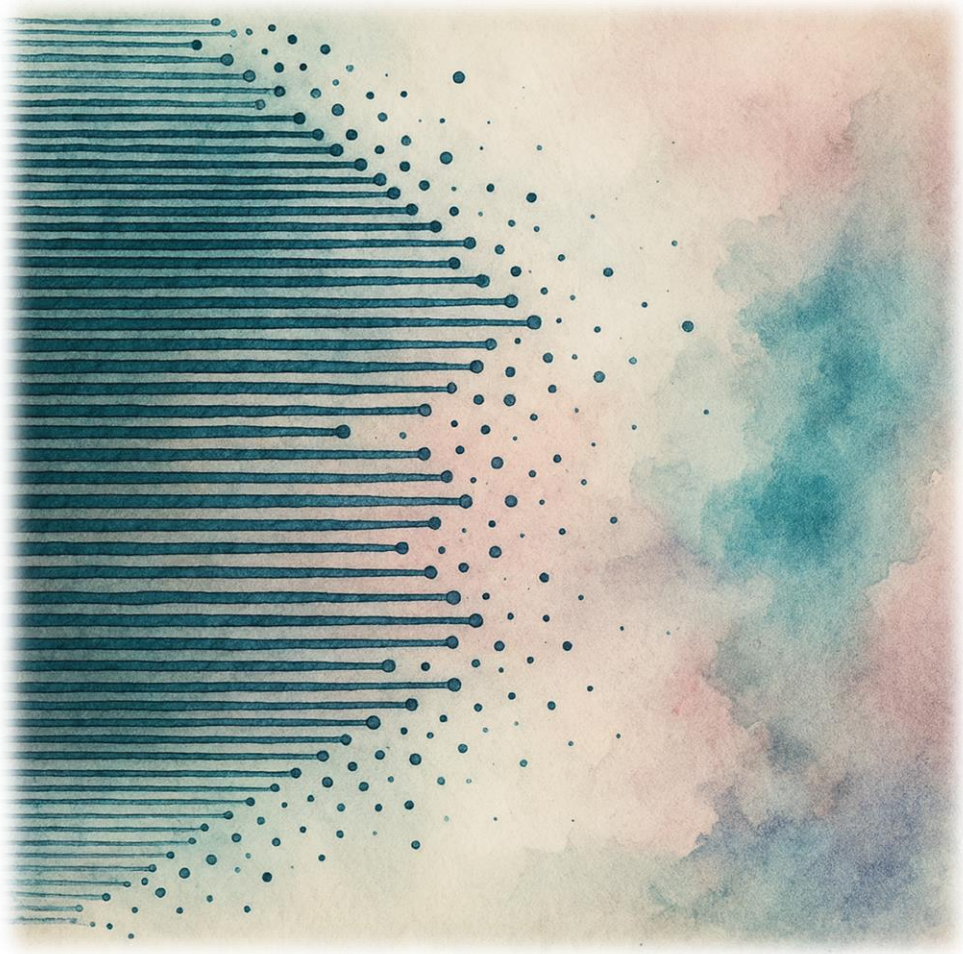
CAVE:

k-Means may have different clustering outcomes which are all correct!

```r
# Load necessary libraries
library(ggplot2)

# Define six points in 2D space
points_2d <- data.frame(
  Point = paste0("Sample_", 1:6),
  x = c(0.7, -0.5, 0.2, -0.3, 0.8, -0.7),
  y = c(0.5, -0.7, 0.3, 0.6, -0.4, 0.1)
)

# Set the number of clusters (k)
k <- 2

# Perform k-means clustering
set.seed(42)  # Ensure reproducibility
kmeans_result <- kmeans(points_2d[, c("x", "y")], centers = k, nstart = 10)

# Add cluster assignments to the original data
points_2d$Cluster <- as.factor(kmeans_result$cluster)

# Extract centroid coordinates
centroids <- as.data.frame(kmeans_result$centers)
colnames(centroids) <- c("x", "y")
centroids$Cluster <- as.factor(1:k)

# Visualize the clustering results
ggplot(points_2d, aes(x = x, y = y, color = Cluster)) +
  geom_point(size = 4) +  # Points with cluster colors
  geom_text(aes(label = Point), vjust = -1.5, size = 6) +  # Add labels
  geom_point(data = centroids, aes(x = x, y = y),
             size = 6, shape = 4, color = "black") +  # Centroids
  xlim(-2, 2) + ylim(-2, 2) +  # Set axis limits
  theme_minimal(base_size = 18) +
  labs(
    title = "K-Means Clustering of Six Points",
    x = "X-axis",
    y = "Y-axis",
    color = "Cluster"
  ) +
  theme(plot.title = element_text(hjust = 0.5))
```

1. Initialize *k* centroids at random positions
2. Assign each sample to the closest centroid
3. Calculate the mean position of the points of each cluster
4. Move the centroids to this position
5. Repeat 2.
6. Finish, if:
   a.) Convergence
   b.) Max. iteration number

Massive Parallel Hypothesis Testing



Linear Models for Microarray Data (LIMMA)

Massive Parallel Hypothesis Testing

gene	Sample_1-1	Sample_1-2	Sample_1-3	Sample_1-4	Sample_2-1	Sample_2-2	Sample_2-3	Sample_2-4
Gene_0001	131	163	379	513	173	348	144	241
Gene_0002	36	86	46	29	26	67	68	58
Gene_0003	76	136	118	190	53	61	63	61
Gene_0004	99	88	93	110	358	249	282	
Gene_0005	177	108	143	146	130	150	79	57

- thousands of genes are tested at once → fit **one linear model per gene** and **adjust p-values**
- **design matrix** encodes the experiment: groups, pairing, and covariates
- always correct for multiple testing and **report both effect size (logFC) and FDR**

Understanding logFCs

gene	Sample_1-1	Sample_1-2	Sample_1-3	Sample_1-4	Sample_2-1	Sample_2-2	Sample_2-3	Sample_2-4
Gene_0001	131	163	379	513	173	348	144	241
Gene_0002	36	86	46	29	26	67	68	58
Gene_0003	76	136	118	190	53	61	63	61
Gene_0004	99	88	93	110	358	249	282	[illegible]
Gene_0005	177	108	143	146	130	150	79	57

$$logFC = log_2(mean\ expression\ of\ group\ 1) - log_2(mean\ expression\ of\ group\ 2)$$

Understanding logFCs

gene	Sample_1-1	Sample_1-2	Sample_1-3	Sample_1-4	Sample_2-1	Sample_2-2	Sample_2-3	Sample_2-4
Gene_0001	131	163	379	513	173	348	144	241
Gene_0002	36	86	46	29	26	67	68	58
Gene_0003	76	136	118	190	53	61	63	61
Gene_0004	99	88	93	110	358	249	282	
Gene_0005	177	108	143	146	130	150	79	57

$$logFC = log_2(mean\ expression\ of\ group\ 1) - log_2(mean\ expression\ of\ group\ 2)$$

Understanding logFCs

gene	Sample_1-1	Sample_1-2	Sample_1-3	Sample_1-4	Sample_2-1	Sample_2-2	Sample_2-3	Sample_2-4
Gene_0001	131	163	379	513	173	348	144	241
Gene_0002	36	86	46	29	26	67	68	58
Gene_0003	76	136	118	190	53	61	63	61
Gene_0004	99	88	93	110	358	249	282	
Gene_0005	177	108	143	146	130	150	79	57

$$logFC = log_2(mean\ expression\ of\ group\ 1) - log_2(mean\ expression\ of\ group\ 2)$$

$$logFC = log_2\left(\frac{1}{4}(131 + 163 + 379 + 513)\right) - log_2\left(\frac{1}{4}(173 + 348 + 144 + 241)\right)$$

Understanding logFCs

gene	Sample_1-1	Sample_1-2	Sample_1-3	Sample_1-4	Sample_2-1	Sample_2-2	Sample_2-3	Sample_2-4
Gene_0001	131	163	379	513	173	348	144	241
Gene_0002	36	86	46	29	26	67	68	58
Gene_0003	76	136	118	190	53	61	63	61
Gene_0004	99	88	93	110	358	249	282	
Gene_0005	177	108	143	146	130	150	79	57

$$logFC = log_2(mean\ expression\ of\ group\ 1) - log_2(mean\ expression\ of\ group\ 2)$$

$$logFC = log_2\left(\frac{1}{4}(131 + 163 + 379 + 513)\right) - log_2\left(\frac{1}{4}(173 + 348 + 144 + 241)\right)$$

$$\underline{logFC = 0.3885}$$

Understanding logFCs

gene	Sample_1-1	Sample_1-2	Sample_1-3	Sample_1-4	Sample_2-1	Sample_2-2	Sample_2-3	Sample_2-4
Gene_0001	131	163	379	513	173	348	144	241
Gene_0002	36	86	46	29	26	67	68	58
Gene_0003	76	136	118	190	53	61	63	61
Gene_0004	99	88	93	110	358	249	282	
Gene_0005	177	108	143	146	130	150	79	57

- **Symmetry**: logFC of +1 means A is two-fold B; −1 means A is half-fold B.
- **Additive** on the log scale, **multiplicative** on the raw scale.
- Reference matters: always state the contrast explicitly (A vs B); **sign flips when you swap**

LIMMA

```r
library(readr)
library(limma)
library(edgeR)

# 1) Load data
expr_df <- readr::read_tsv("expression_matrix.tsv")      # gene + count co
pheno   <- readr::read_tsv("pheno.tsv")                  # sample_id, grou

# 2) Shape matrix and align columns to pheno
gene_ids <- expr_df$gene
expr_mat <- as.matrix(expr_df[ , -1, drop = FALSE])
rownames(expr_mat) <- gene_ids

stopifnot(all(colnames(expr_mat) %in% pheno$sample_id))
pheno <- pheno[match(colnames(expr_mat), pheno$sample_id), , drop = FALSE]
stopifnot(identical(colnames(expr_mat), pheno$sample_id))

# 3) Design matrix
pheno$group <- factor(pheno$group)  # ensure factor
if ("batch" %in% colnames(pheno)) {
  design <- stats::model.matrix(~ 0 + group + batch, data = pheno)
} else {
  design <- stats::model.matrix(~ 0 + group, data = pheno)
}
colnames(design) <- make.names(colnames(design))
```

```r
# 4) voom + linear modeling
y <- edgeR::DGEList(counts = expr_mat)
keep <- edgeR::filterByExpr(y, group = pheno$group)
y <- y[keep, , keep.lib.sizes = FALSE]
y <- edgeR::calcNormFactors(y, method = "TMM")

v   <- limma::voom(y, design = design, plot = FALSE)
fit <- limma::lmFit(v, design)

# 5) Contrast: last level vs first level
lev <- levels(pheno$group)
contrast_str <- paste0("group", lev[2], " - group", lev[1])
cont <- limma::makeContrasts(contrasts = contrast_str, levels = design)

fit2 <- limma::contrasts.fit(fit, cont)
fit2 <- limma::eBayes(fit2, robust = TRUE, trend = TRUE)

# 6) Results
tt <- limma::topTable(fit2, coef = 1, number = Inf, sort.by = "P")
tt <- cbind(gene = rownames(tt), tt, FDR = stats::p.adjust(tt$P.Value, method = "BH"))
rownames(tt) <- NULL

# 7) Save and quick summary
readr::write_tsv(as.data.frame(tt), "limma_results.tsv")
print(head(tt, 10))
cat("Significant at FDR < 0.05:", sum(tt$FDR < 0.05, na.rm = TRUE), "\n")
```

LIMMA

	gene	logFC	AveExpr	t	P.Value	adj.P.Val	B	FDR
1	Gene_0339	-2.21 5204	9.035129	-6.088732	1.208 7e-09	1.202 2e-06	11.57961209	1.20 2872e-06
2	Gene_0915	2.0322443	10.268993	5.881434	4.287 8e-09	2.133 0e-06	10.40577957	2.13 3110e-06
3	Gene_0210	-2.0147994	9.715177	-5.712519	1.166 3e-08	3.870 8e-06	9.45983021	3.87 0128e-06
4	Gene_0890	1.8837134	10.423634	5.497146	4.019 5e-08	9.999 5e-06	8.30496880	9.99 9315e-06
5	Gene_0367	-1.7047143	11.909221	-4.984784	6.378153e-07	1.269 2e-04	5.72118076	1.26 0252e-04
6	Gene_0070	-1.6633950	10.543575	-4.897397	9.967239e-07	1.514 8e-04	5.30551953	1.51 4028e-04
7	Gene_0488	2.0639220	7.138816	4.884278	1.065145e-06	1.514 8e-04	5.23043401	1.51 4028e-04
8	Gene_0350	-1.6437104	11.543846	-4.844480	1.301486e-06	1.618 3e-04	5.05682470	1.61 3723e-04
9	Gene_0635	1.6140405	10.723104	4.796380	1.654826e-06	1.756		
10	Gene_0015	-1.6474202	10.163370	-4.783353	1.765380e-06	1.756553e-04	4.77579401	1.75 5553e-04
11	Gene_0818	1.6460932	9.995901	4.754757	2.033500e-06	1.839393e-04	4.64517552	1.83 9393e-04
12	Gene_0366	-1.6131175	9.723063	-4.629474	3.743572e-06	3.104045e-04	4.08071882	3.10 4045e-04
13	Gene_0421	-1.5445493	11.011676	-4.611719	4.076833e-06	3.120345e-04	3.99492461	3.12 0345e-04
14	Gene_0943	-1.5448247	11.397701	-4.571935	4.929893e-06	3.130136e-04	3.82044743	3.13 0136e-04
15	Gene_0750	1.8094575	8.078639	4.567343	5.038716e-06	3.130136e-04	3.81625532	3.130136e-04
	58	-1.6626657	8.997802	-4.564523	5.106658e-06	3.130136e-04	3.79930370	3.130136e-04

Figures to Consider in a LIMMA:

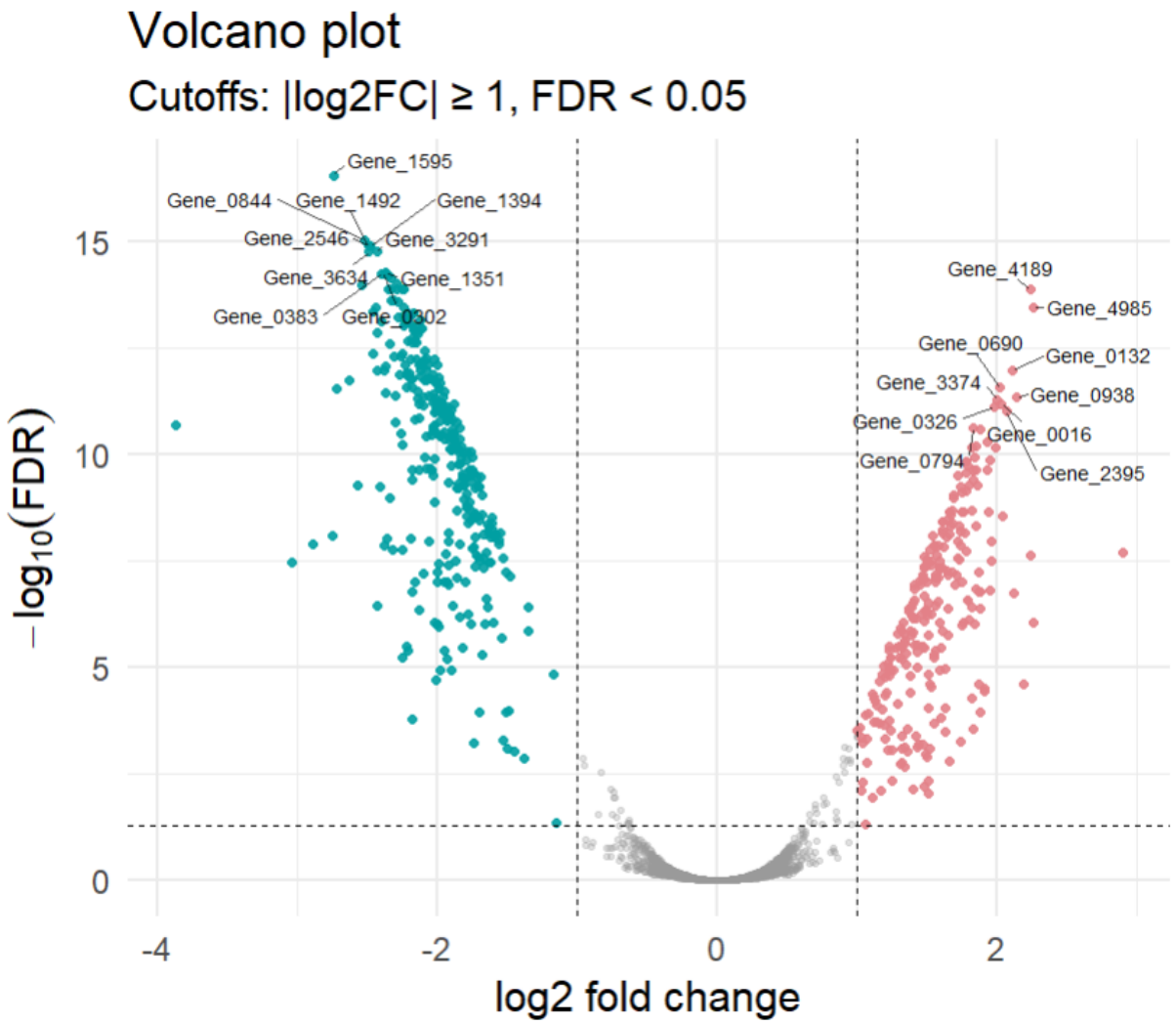
logFC: measure of effect size

P.Value: Is the effect size high enough to account for a non-random effect?

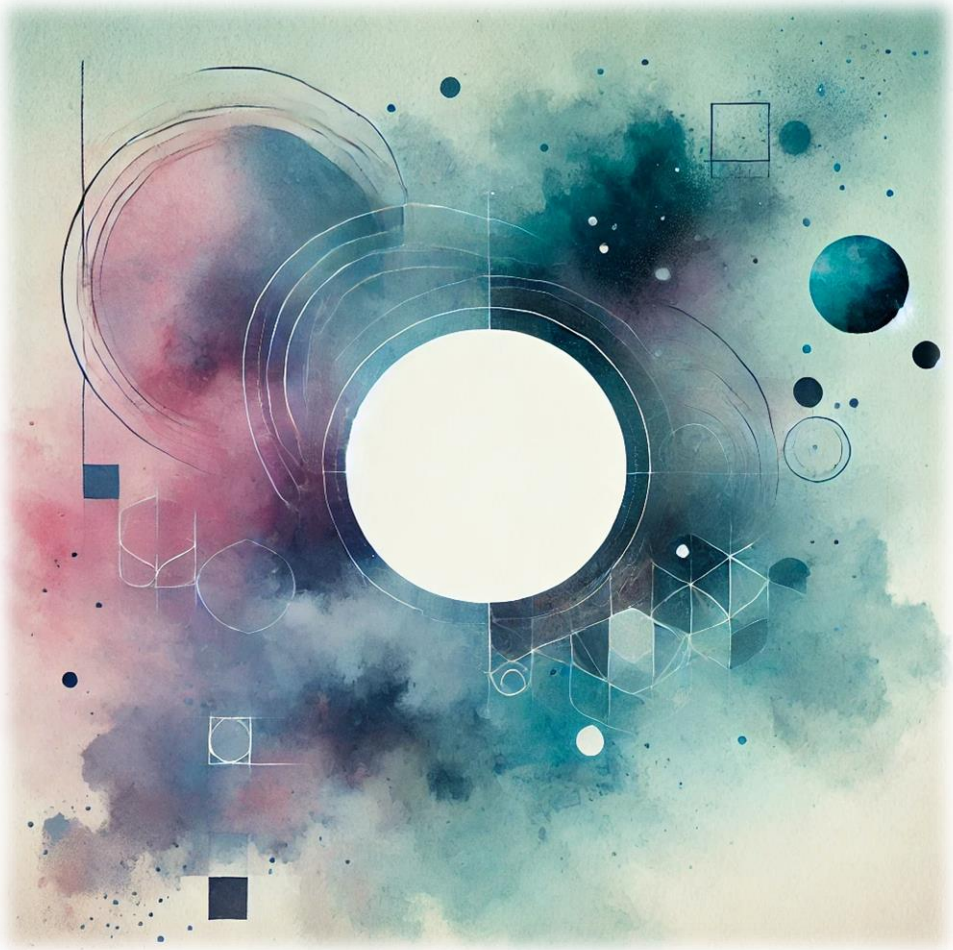
Adj.P.Val: Is the effect size high enough to account for a non-random effect given that I tested some thousand genes?

FDR: Same as adj. p value

LIMMA Visualization



Excursus: Missingness and Data Imputation



Excursus: Missingness and Data Imputation

id	group	x	y	count
1	A	1.2	*NA*	0
2	A	*NA*	5.1	1
3	B	3.4	4.7	*NA*
4	B	2.2	*NA*	4
5	B	*NA*	2.9	2
6	A	1.8	3.3	*NA*

What are NAs?

NAs are "not available" values. They mean the true value is unknown or missing.

Where do NAs come from?

- Measurement failed or below detection limit
- Sample lost or filtered during QC
- Merge of datasets with non-overlapping features
- Manual entry gaps or parsing issues

Excursus: Missingness and Data Imputation

id	group	x	y	count
1	A	1.2	*NA*	0
2	A	*NA*	5.1	1
3	B	3.4	4.7	*NA*
4	B	2.2	*NA*	4
5	B	*NA*	2.9	2
6	A	1.8	3.3	*NA*

Difference between NA and 0?

NA: value is unknown. Could be anything.

0: value is known and equals zero. Very different statistically.

Why are NAs a challenge?

- Many algorithms can't run with missing entries
- They change means, variances, and correlations if handled poorly
- Patterns of missingness can be informative and must be checked

Excursus: Missingness and Data Imputation

id	group	x	y	count
1	A	1.2	*NA*	0
2	A	*NA*	5.1	1
3	B	3.4	4.7	*NA*
4	B	2.2	*NA*	4
5	B	*NA*	2.9	2
6	A	1.8	3.3	*NA*

Difference between NA and 0?

NA: value is unknown. Could be anything.

0: value is known and equals zero. Very different statistically.

Why are 0s a challenge?

- Inflate variance and bias mean estimates when treated as real values.
- Cause mathematical issues in transformations: log(0) → undefined ratios or fold-changes → infinite values
- Lead to false contrasts between groups if zeros are unevenly distributed.

Excursus: Missingness and Data Imputation

id	group	x	y	count
1	A	1.2	*NA*	0
2	A	*NA*	5.1	1
3	B	3.4	4.7	*NA*
4	B	2.2	*NA*	4
5	B	*NA*	2.9	2
6	A	1.8	3.3	*NA*

Data Imputation

- Goal: Estimate missing values based on observed data structure.
- Principle: Replace NAs with plausible estimates to restore analytical completeness.
- Pros:
  - Enables algorithms requiring complete matrices
  - Preserves sample structure for PCA, clustering, regression
- Cons:
  - Adds artificial information → may inflate confidence
  - Choice of method can alter biological interpretation
  - Should always be transparent and justified

Excursus: Missingness and Data Imputation

```r
# ──────────────────────────────────────────────
# Example: kNN Imputation for missing values
# ──────────────────────────────────────────────
if (!requireNamespace("impute", quietly = TRUE))
  BiocManager::install("impute")

library(impute)

# Simulate data with NAs
set.seed(42)
mat <- matrix(rnorm(50), nrow = 10)
mat[sample(length(mat), 5)] <- NA

# kNN imputation (k = 5)
mat_imputed <- impute::impute.knn(mat, k = 5)$data

# Visual comparison
mat[1:5, 1:5]
mat_imputed[1:5, 1:5]
```

Excursus: Missingness and Data Imputation

```r
# ------------------------------------------------------------
# Example: kNN Imputation for missing values
# ------------------------------------------------------------
if (!requireNamespace("impute", quietly = TRUE))
  BiocManager::install("impute")

library(impute)

# Simulate data with NAs
set.seed(42)
mat <- matrix(rnorm(50), nrow = 10)
mat[sample(length(mat), 5)] <- NA

# kNN imputation (k = 5)
mat_imputed <- impute::impute.knn(mat, k = 5)$data

# Visual comparison
mat[1:5, 1:5]
mat_imputed[1:5, 1:5]
```

```
> mat[1:5, 1:5]
           [,1]       [,2]       [,3]      [,4]       [,5]
[1,]  1.3709584  1.3048697 -0.3066386 0.4554501  0.2059986
[2,] -0.5646982  2.2866454 -1.7813084 0.7048373 -0.3610573
[3,]  0.3631284 -1.3888607 -0.1719174 1.0351035  0.7581632
[4,]  0.6328626 -0.2787888         NA -0.6089264 -0.7267048
[5,]         NA -0.1333213  1.8951935 0.5049551 -1.3682810
```

Excursus: Missingness and Data Imputation

```r
# ------------------------------------------------------------
# Example: kNN Imputation for missing values
# ------------------------------------------------------------
if (!requireNamespace("impute", quietly = TRUE))
  BiocManager::install("impute")

library(impute)

# Simulate data with NAs
set.seed(42)
mat <- matrix(rnorm(50), nrow = 10)
mat[sample(length(mat), 5)] <- NA

# kNN imputation (k = 5)
mat_imputed <- impute::impute.knn(mat, k = 5)$data

# Visual comparison
mat[1:5, 1:5]
mat_imputed[1:5, 1:5]
```

```
> mat[1:5, 1:5]
           [,1]       [,2]       [,3]      [,4]       [,5]
[1,]  1.3709584  1.3048697 -0.3066386 0.4554501  0.2059986
[2,] -0.5646982  2.2866454 -1.7813084 0.7048373 -0.3610573
[3,]  0.3631284 -1.3888607 -0.1719174 1.0351035  0.7581632
[4,]  0.6328626 -0.2787888         NA -0.6089264 -0.7267048
[5,]         NA -0.1333213  1.8951935 0.5049551 -1.3682810
```

```
mat_imputed[1:5, 1:5]
           [,1]       [,2]       [,3]       [,4]       [,5]
[1,]  1.3709584  1.3048697 -0.3066386  0.4554501  0.2059986
[2,] -0.5646982  2.2866454 -1.7813084  0.7048373 -0.3610573
[3,]  0.3631284 -1.3888607 -0.1719174  1.0351035  0.7581632
[4,]  0.6328626 -0.2787888  0.0521643 -0.6089264 -0.7267048
[5,]  0.7631515 -0.1333213  1.8951935  0.5049551 -1.3682810
```

frank.hause@medizin.uni-halle.de

@fhause.bsky.social